Realising adversarial self-testing

VER: $\alpha.1$

Abstract

Self-testing allows a classical referee to deduce the inner workings of an untrusted quantum device consisting of two or more non-communicating parts. Consider the bipartite case, where Alice and Bob each holds one part of the quantum device. If Alice and Bob trust each other, then using known results, they can self-test their joint quantum device. However, we consider the setting where they do not trust each other. This 'adversarial self-testing' setup was introduced by BAHS '20 who showed that it is impossible to achieve it with information theoretic security—even when the parties are allowed to run an interactive protocol. Here, we complete the picture for the bipartite setting and show how to achieve adversarial self-testing under standard computational assumptions. To this end, we formalise secure two party device independent computations and define adversarial self-testing and non-local games as ideal functionalities and give a composably-secure construction assuming ideal commitments.

Contents

1 Introduction **3**

2 Preliminaries **3**
2.1 Notation . . . 3
2.2 Non-local games . . . 3
2.3 Standard MPQC . . . 3
2.3.1 Functionalities . . . 3
2.3.2 Two Party Protocols . . . 3
2.3.3 Define QIM . . . 4
2.3.4 Indistinguishability . . . 4
2.3.5 QSA and UC security . . . 4

I DI-2PC **5**

3 DI-2PC definition **5**

4 Impossibility of DI-2PC with local isometries **7**

5 Definition of 2-FAST **7**
5.1 Device Notation . . . 7
5.2 2-FAST functionality . . . 8
5.3 Impossibility of 2-FAST with information-theoretic security . . . 9
5.4 $\mathcal{F}_{\mathsf{NL}}$ functionality . . . 9

6 Protocols for 2-FAST and $\tilde{\mathcal{F}}_{\mathsf{NL}}$ **10**
6.1 Protocol for $\tilde{\mathcal{F}}_{\mathsf{NL}}$ from commitments . . . 10
6.2 Protocol for 2-FAST from $\tilde{\mathcal{F}}_{\mathsf{NL}}$ . . . 15

7 Application: Coin flipping with everlasting security **20**
7.1 Impossibility of coin flipping in the plain model . . . 21
7.2 Coin flipping in the shared randomness model . . . 22
7.3 Coin flipping in the shared-EPR model . . . 26
7.3.1 Computing predictability of measure-and-compute is an SDP . . . 28
7.3.2 Stating the measure-and-XOR protocol in the SDP formalism . . . 32
7.4 Towards simulation security for coin flipping . . . 33

II Device Independent Multi-Party Computation | DI MPC **34**

8 DI MPC definition **34**

9 Relation to prior work **34**

10 Protocol for DI MPC with quantum communication **34**

11 Definition m-FAST **34**

12 Protocol for m-FAST under a compiled rigidity conjecture **34**

1 Introduction

2 Preliminaries

In this section we summarise some of the notation used in this work together with some facts from self-testing, non-local games, classical and quantum multiparty computation.

2.1 Notation

We will denote as a quantum register a collection of qubits that we group together as a single unit. We will denote most registers using uppercase letters A, B, etc. Each register has associated a Hilbert space which we will often denote using capital script letters such as $\mathcal{H}, \mathcal{K}$, etc. For given Hilbert spaces $\mathcal{H}, \mathcal{K}$, we let $\mathsf{L}(\mathcal{H}, \mathcal{K})$ be the set of all linear mappings (or operators) from $\mathcal{H}$ to $\mathcal{K}$. We denote as $\mathsf{Pos}(\mathcal{H})$ the set of all positive semidefinite operators acting on $\mathcal{H}$ and $\mathsf{D}(\mathcal{H})$ the set of unit trace, positive semidefinite operators (also called density matrices) acting on $\mathcal{H}$. The identity operator in $\mathsf{L}(\mathcal{H})$ is denotes as $\mathbb{I}_{\mathcal{H}}$. Sometimes we abuse notation and do not make a distinction between the Hilbert space $\mathcal{H}$ and the register A to which it corresponds.

2.2 Non-local games

Definition 1 ((Bipartite) Non-local game)**.** *A* non-local game *for two players is defined by the tuple* $G := (A, B, X, Y, \mathcal{D}, \mathsf{pred})$ *where* A, B, X, Y *are finite sets,* $\mathcal{D}$ *is a product (probability) distribution over* X *and* Y*, and* $\mathsf{pred} : A \times B \times X \times Y \to \{0, 1\}$ *is a predicate that indicates whether the players win by responding with* $(a, b) \in A \times B$ *to questions* $(x, y) \in X \times Y$*. The* winning probability *for the game is*

$$\Pr(\text{win}) = \sum_{a,b,x,y \in A \times B \times X \times Y} \mathsf{pred}(abxy) \Pr(ab|xy) \Pr(xy)$$

where $\Pr(xy)$ *is determined by* $\mathcal{D}$ *and* $\Pr(ab|xy)$ *is the strategy used by Alice and Bob to answer the questions.*

The game G is interpreted as follows: a referee sends questions (x, y) to Alice and Bob respectively, sampled according to $\mathcal{D}$, and the players respond with (a, b) respectively. The most general *classical strategy* for Alice and Bob is given by

$$\Pr(ab|xy) = \sum_{\lambda} \Pr(a|x\lambda) \Pr(b|y\lambda) \Pr(\lambda)$$

while the most general *quantum strategy* is given by

$$\Pr(ab|xy) = \langle\psi| M_{a,x} \otimes M_{b,x} |\psi\rangle$$

where $|\psi\rangle_{\mathcal{H}_{AB}}$ is a bipartite state[1] on $\mathcal{H}_{AB} = \mathcal{H}_A \otimes \mathcal{H}_B$ shared by Alice and Bob while $M_{a,x}$ (resp. $M_{b,y}$) denote Alice (resp. Bob) using a local measurement indexed by x (resp. y) and obtaining outcome a (resp. b).

It turns out that if a strategy wins with the highest possible (quantum) probability in certain non-local games, such as the CHSH game, then the *device specification* $\mathsf{spec} = (|\psi\rangle_{\mathcal{H}_{AB}}, \{M_x\}_x, \{M_y\}_y)$ is fixed up to local isometries. This phenomenon is termed self-testing—the classical input output behaviour of the device *self-tests* the device's specification. In fact, for every bipartite entangled state, one can construct such a self-test.

Theorem 2 (informal: TODO: write the formal statement)**.** *Every bipartite entangled state can be self-tested, i.e. for every bipartite state* $|\psi\rangle$*, one can find a device specification which can be self-tested.*

2.3 Standard MPQC

2.3.1 Functionalities

2.3.2 Two Party Protocols

Definition 3 (Quantum Turing Machine and Quantum Interactive Time Machines)**.** *(TODO: Define it formally.) Denote by* QTM *a uniform poly sized quantum circuit that takes an input and produces an output (the input output are*

[1] The notation is slightly degenerate: we use A, B to refer to Alice and Bob's system and AB for their respective set of answers.

allowed to be quantum, depending on the application). Denote by QIM *a uniform poly sized quantum circuit that takes an input, interacts with other quantum circuits, and produces an output.*

The details of how the interaction happens can be formalised but it is not particularly insightful.[2]

2.3.3 Define QIM

2.3.4 Indistinguishability

Definition 4. *Two* QTM*s,* M_0 *and* M_1*, are* (t, ϵ)*-indistinguishable, if for any* $t(n)$*-time* QTM Z *and any mixed state* $\sigma_n \in \mathcal{H}_{\mathsf{n}} \otimes \mathcal{R}_{\mathsf{n}}$*, where* $\mathcal{H}_n$ *is the register for the input of* QTM*s* M_0, M_1 *and* $\mathcal{R}_n$ *is an auxiliary register,*

$$\left|\Pr\left[Z((M_0 \otimes \mathbb{I}_{\mathsf{L}(\mathcal{R})})\sigma_n) = 1\right] - \Pr\left[Z((M_0 \otimes \mathbb{I}_{\mathsf{L}(\mathcal{R})})\sigma_n) = 1\right]\right| \leq \epsilon(n). \tag{1}$$

Remark 5. We say M_0 and M_1 are quantum computationally indistinguishable, denoted as $M_0 \approx_{\mathsf{qc}} M_1$, when for every polynomial $t(n)$ there is a negligible $\epsilon(n)$ such that M_0 and M_1 are (t, ϵ)-indistinguishable.

Notation 6. We use the following two notations:

- $M_0 \approx_{\mathsf{qc}} M_1$: For any two QTMs, M_0 and M_1, it means that $\forall$ QTMs $(Z_{\mathsf{in}}, Z_{\mathsf{out}}) =: Z$, it holds that

  $$|\Pr[1 \leftarrow \langle M_0, Z\rangle] - \Pr[1 \leftarrow \langle M_1, Z\rangle]| \leq \mathsf{negl}$$

  where $\langle M_b, Z\rangle$ interact
- $M_0 \approx_{\mathsf{qci}} M_1$: For any two QIMs, M_0 and M_1

2.3.5 QSA and UC security

Definition 7. *Let* Π *and* Γ *be two polynomial time protocols. The protocol* Π *QSA-realises* Γ *if for any poly-time* QIM $\mathcal{A}$ *there is another poly-time* QIM $\mathcal{S}$ *such that* $M_{\Pi,\mathcal{A}} \approx_{\mathsf{qc}} M_{\Gamma,\mathcal{S}}$.

Definition 8. *Let* Π *and* Γ *be two polynomial time protocols. The protocol* Π *UC-realises* Γ *if for any poly-time* QIM $\mathcal{A}$ *there is another poly-time* QIM $\mathcal{S}$ *such that* $M_{\Pi,\mathcal{A}} \approx_{\mathsf{qci}} M_{\Gamma,\mathcal{S}}$.

[2] To avoid any confusion, we include the formal definition in Definition **??**.

Part I
DI-2PC

3 DI-2PC definition

Before we consider device independent two party computations, we first formally define the notion of a classical party having access to an untrusted quantum device. [3]

Definition 9 (Device Independent Machine)**.** *A Device Independent Machine (*DM*) is a pair $D = (M_c, M_q)$ consisting of an interactive polynomial-time TM, M_c, and a universal* QIM*, M_q. The machine M_c has an input register S_{in}, an ancilla register S_{anc} and an output register S_{out}. All this registers are classical. Additionally, there's a classical interaction register S_{int} which M_c uses to communicate with M_q. The* QIM *M_q has a quantum input register $\mathcal{H}$, ancilla register $\mathcal{H}_{anc}$ and output register $\mathcal{H}_{out}$. Moreover, it has access to an additional special input register $\mathcal{H}_D$ (which we denote as the special device input).*

[Can we say a bit why $\mathcal{H}_D$ consists of two parts $\mathcal{H}_{D,1}$ and $\mathcal{H}_{D,2}$?] better to make it explicit when detailing M_q

The register $\mathcal{H}_D$ is defined in order to hold in memory a strategy for the device M_q. Since this input can be modified by the adversary at the start of a protocol, then the device is untrusted from the point of view of a honest M_c. In our protocols we will consider M_q with a fixed behaviour and assume that the adversary is not capable of corrupting M_q itself except by modifying the inputs. Therefore, once the input is processed by M_q, the adversary will not be able to change its behaviour arbitrarily. We could instead have defined two separate parties, one of them classical and the other quantum. In this case, the adversary could choose to corrupt the quantum party, thus making the quantum device behave in an arbitrary manner. In particular, the adversary could make different devices coordinate their answers while the honest classical party believes its performing local measurements.

Following the previous discussion, in this work we will consider DMs where M_q in Definition 9 has a fixed program and the strategy that the device will follow will be given by the classical device input. We proceed to define this fixed machine. When a DM $D = (M_c, M_q)$ is activated, the classical machine executes its program first and asks M_q to execute a measurement on some set of qubits by writing its description on the interaction register, activating M_q. When M_q starts its first round, it measures a part of the device input which is decomposed into two registers $\mathcal{H}_D = \mathcal{H}_{D,1} \otimes \mathcal{H}_{D,2}$. Concretely, it measures $\mathcal{H}_{D,1}$ in the computational basis and interprets the measured string as a QTM M_S which corresponds to a strategy that M_q will follow. Then, M_q runs M_S with a classical input corresponding to the transcript between M_c and M_q. The output of M_S is interpreted as a circuit together with measurements acting on its input H_{in}, ancilla $\mathcal{H}_{anc}$ and $\mathcal{H}_{D,2}$ register. Having obtained some measurement results, M_q runs M_S with these results and the output of M_S corresponds to a string which M_q passes to M_c through the interaction register. After this, M_c is activated again, which has the choice to continue the interaction or activating another machine.

We assume that if the classical part of the special input is not well-formed then M_q just runs the instructions as M_c intends. If the DM is also interacting with other machines in a multiparty protocol, the definition can be modified to include interaction registers with these other parties. In this case, classical communication with other parties is managed by M_c.

Sometimes, a classical party will be assumed to be able to "partition" its quantum devices into several parts which can't communicate with each other. I can't remember what this requirement was for. [I am not sure but I would guess because we were hoping to do a full blown MPC and had to somehow make sure local DI quantum computations can be performed by honest parties]

Definition 10. *s*

When M_q is first activated, the machine receives an input in the $\mathcal{H}_D$ register. We can assume that the quantum state in this register has the following specific form.

Definition 11. *A quantum device Ψ is a classical-quantum state on a k-partite register (or Hilbert space) $\square_{A_1,\cdots,A_k} = \square_{A_1} \otimes \cdots \otimes \square_{A_k}$ where each $\square_{A_i}$ consist of a classical and a quantum register, i.e. $\square_{A_i} = S_i \otimes \mathcal{H}_i$ where S_i holds a string describing measurements and $\mathcal{H}_i$ holds quantum states. The quantum registers across $\square_{A_1} \otimes \cdots \square_{A_k}$ may*

[3] Self-note: There's also this paper defining a notion of classically controlled quantum computation https://arxiv.org/abs/quant-ph/0407008

contain an entangled state. The classical registers S_i contain the description of a quantum turing machine M. Therefore, a quantum device $\Psi = |\Psi\rangle\langle\Psi|$ is of the following form

$$|\Psi\rangle_{\square_{A_1,\cdots,A_k}} = |\mathsf{str}(M_1)\rangle \otimes \cdots \otimes |\mathsf{str}(M_k)\rangle \otimes |\psi\rangle_{A_1,\cdots,A_k}.$$

where $\mathsf{str}(M_i)$ *is a string describing the QTM* M_i*.*

[Indeed, the multi party thing needs to be defined a bit more formally and carefully—not just as a small comment. Maybe we can look at the HSS paper for inspiration?]

yeah, I have not written much because I'm trying to understand something about the inputs. Let me know when you get some time to talk.

[Why are we placing restrictions (such as Clifford+T, and the set I) on how M_q and M_c interact? Shouldn't that be specified later by whatever protocol we choose to run? Let's chat over Zoom etc. when convenient.]

A k-party device independent protocol $\Pi = (D_1, D_2, \cdots, D_k)$ is a tuple of k DMs. The protocols we consider will be under attack by an adversary which we usually denote $\mathcal{A}$. An adversary is a QTM which can corrupt parties during the execution of a protocol. We say a protocol is poly-time when the parties involved run in polynomial time. We often don't make a distinction between a party and the machine that the party is running, although in the case of DMs we sometimes refer to the classical machine as the party and the quantum machine as the device of the party.

Execution of protocol When executing a protocol $\Pi = (D_1, D_2, \cdots, D_k)$ against an adversart $\mathcal{A}$ we mostly follow the conventions in [HSS15]. We give a summary of these and add some considerations when the parties are DMs. At the begining of the execution all parties start with their corresponding inputs in the input registers which include a description of the security parameter in unary 1^λ. The first party to activate is the adversary $\mathcal{A}$ and guides the order of execution for each party.

The adversary can choose to either corrupt a party or deliver a message. Corrupting a party consists in making the corrupted party run an arbitrary program chosen by the adversary. When a party becomes corrupted, it gives the content in its registers to the adversary, which chooses what program to run based on the input. Once a party becomes corrupted, all its history becomes known to $\mathcal{A}$. When corrupting a party corresponding to a DM, this always means that the classical party is corrupted. We do not allow the adversary to corrupt only the quantum device. Delivering messages consists in choosing a corrupted party P and making P send a message through some communication register. We assume that all communication registers are authenticated.

When a party gets activated by $\mathcal{A}$, it runs its machine and at then writes in the communication registers the corresponding messages computed from its inputs. Once the party writes a message, $\mathcal{A}$ gets activated and coordinates the message delivery. At some point of the protocol the party decides to write an outputs and writes it in the output register. When the party is a DM and gets activated, the classical party starts by processing the input and coordinating the message delivery between itself and its quantum device, once it writes a message in the communication register to other party, then $\mathcal{A}$ gets activated and coordinates message delivery.

When running a protocol Π with adversary $\mathcal{A}$, the whole model can be naturally understood as a QIM $M_{\Pi,\mathcal{A}}$. Following the MPC definition of security, we define a secure protocol by realizing an ideal protocol which gives the desired security guarantees. The notion of some protocol realizing another one depends on the indistinguishability of the outputs produced by the protocol and the idealized version. Following the notation for indistinguishability in MPC, we define a notion of indistuinguishability for DMs.

Indistinguishability Quantum computational indistinguishability between two machines M_1 and M_2 is defined in terms of a quantum machine (the distinguisher) which chooses the inputs to each of them and then decides based on the outputs which machine is it interacting with. When trying to distinguish IDMs, the distinguisher interacts only with the classical part and not the devices. Given a QIM Z and a machine (either QIM or IDM) M, we denote as $\langle Z(\sigma), M\rangle$ the procedure involving giving σ as input to Z which then provides an input to M and interacts with M. The definition for computation indistinguishability in the case of IDMs follows the definition given for QIMs This definition was written below as a "Notation" perhaps its better to write it as a definition.

As already mentioned, We will consider DMs where M_q in Definition 9 has a fixed program and the strategy that the device will follow will be given by the classical device input. We proceed to define this fixed machine.

show that definitions for indistiguinshability work out

Give Ideal functionality for DI-2PC. We should consider one for computing BQP-functions as in Coladangelo et al.

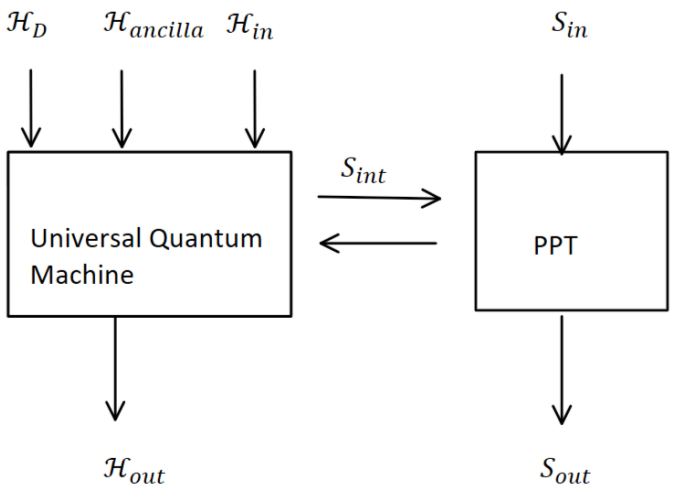


Figure 1: placeholder

Protocol 1 Ideal DI-2PC protocol

Global inputs. A description of a quantum circuit Q acting on $\mathcal{H}_A \otimes \mathcal{H}_B \otimes \mathcal{H}_{anc}$.

Inputs. A receives $\square_A \in \mathcal{H}_A$, B receives $\square_B \in \mathcal{H}_B$.

The protocol:

1. Each party sends their input directly to the trusted party.
2. Trusted party creates computational zero states $|0\rangle$ in register $\mathcal{H}_{anc}$ (as many as the computation of Q requires).
3. Trusted party applies quantum circuit Q (including measurements) on registers $\mathcal{H}_A \otimes \mathcal{H}_B \otimes \mathcal{H}_{anc}$ and obtaining output registers $\mathcal{H}_A^{out}$, $\mathcal{H}_B^{out}$ and $s_{out} \in \{0, 1\}$.
4. Trusted party sends the register $\mathcal{H}_A^{out}$ and s_{out} to party A and waits for response.
5. If Party A responds with `abort`, the trusted party outputs `abort` to B. If party A responds with any other message, then the trusted party sends register $\mathcal{H}_B^{out}$ and s_{out} to party B.

---

4 Impossibility of DI-2PC with local isometries

5 Definition of 2-FAST

5.1 Device Notation

we need to use different notation for measurements M_x and Turing machines Another way to interpret the game G is in the so-called *device independent* setting where Alice and Bob trust their classical computations but not their quantum computations. For non-local games, one can restrict to *quantum devices* comprising of two parts, each of which is used once, i.e. they take classical inputs and produce classical outputs. More concretely, in this setting, one can suppose that Alice and Bob are given a *quantum device* which consists of two *boxes*. Alice is given Box A and Bob is given Box B. These boxes correspond to some arbitrary device specification $(|\psi\rangle_{\mathcal{H}_{AB}}, \{M_x\}_x, \{M_y\}_y)$. Concretely, Box A contains system $\mathcal{H}_A$ of $|\psi\rangle_{\mathcal{H}_{AB}}$ and on input x, it measures the system using M_x and returns

the outcome a. Similarly Box B measures M_y on system $\mathcal{H}_B$ of $|\psi\rangle_{\mathcal{H}_{AB}}$ and returns the outcome b. Using classical communication, Alice and Bob can estimate $\Pr(\text{win})$ in G by using their respective boxes. From self-testing results, they can, for instance, conclude that their device contained a maximally entangled state and from monogamy of entanglement, they can be sure that nobody else could be entangled with their devices—even if the device itself was prepared by an adversary.

We formalise a quantum device as a state on bipartite registers $\square_{AB}$ as follows.

Notation 12. A *(pure) quantum device* Ψ is a classical quantum state on a bipartite register (or Hilbert space) $\square_{AB} = \square_A \otimes \square_B$ where both $\square_A$ and $\square_B$ consist of a classical and a quantum register, i.e. $\square = S \otimes \mathcal{H}$ where S holds a string describing measurements and $\mathcal{H}$ holds quantum states. The quantum registers across $\square_A \otimes \square_B$ may contain an entangled state while the classical registers contain some canonical classical encoding of measurements $\{M_x\}_x$ and $\{M_y\}_y$. Concretely, corresponding to a game $G = (X, Y, A, B, \mathsf{pred})$, a quantum device $\Psi = |\Psi\rangle\langle\Psi|$ is of the following form

$$|\Psi\rangle_{\square_{AB}} = |\mathsf{string}(\{M_x\}_{x\in X})\rangle_{S_A} \otimes |\mathsf{string}(\{M_y\}_{y\in Y})\rangle_{S_B} \otimes |\psi\rangle_{\mathcal{H}_{AB}}$$

where $M_x = \sum_{a\in A} \Pi_{a|x}$ where $\Pi_{a|x}$ is a projector, and similarly $M_y = \sum_{b\in B} \Pi_{b|y}$ for each a, b, x, y.silly question: Is it correct to write M_x as just the sum of projectors? shouldn't the projectors just sum to unity?

We restrict to pure quantum devices for simplicity. The analysis readily generalises to mixed states.

Notation 13. Let Ψ be a quantum device on $\square_{AB}$ as above, corresponding to a game $\mathsf{G} = (X, Y, A, B, \mathsf{pred})$. We use

- $a \leftarrow \mathsf{MeasureA}_x(\Psi)$ a canonical circuit that acts non-trivially only on register $\square_A$ as follows: it measures the quantum state in system $\square_A$ using M_x and returns the output, a, similarly,
- $b \leftarrow \mathsf{MeasureB}_y(\Psi)$ a canonical circuit that acts non-trivially only on register $\square_B$ as follows: it measures the quantum state in system $\square_B$ using M_y and returns the output b and finally
- $(a, b) \leftarrow \mathsf{MeasureAB}_{xy}(\Psi)$ the outcomes produced by executing both of the above.

Remark 14. There may be cases when the circuit defined associated to $\mathsf{MeasureA}_x$ receives an input where the register S_A does not correspond to a description of a set of measurements $\mathsf{string}(\{\mathsf{M}_x\}_x)$. In such cases, we assume that the circuit simply measures some fixed standard set of measurements.

Remark 15. The registers $\square_A$ and $\square_B$ can be decomposed in several registers, i.e., $\square_A = \square_{A_1,\cdots,A_n} = \square_{A_1} \otimes \cdots \otimes \square_{A_n}$ and $\square_B = \square_{B_1,\cdots,B_n} = \square_{B_1} \otimes \cdots \otimes \square_{B_n}$. In such case we assume the variables $x \in X$ and $y \in Y$ represent measurement choices in each corresponding subsystem.

In practice, we will be interested in self-testing states in a robust manner, i.e., self testing up to some distance δ.

Notation 16. A quantum device $|\Psi\rangle_{AB} = (|\psi\rangle_{AB}, \{\mathsf{M}_x\}_x, \{\mathsf{M}_y\}_y)$ is said to be δ-locally isometric to some specification $\mathsf{spec} = (|\psi'\rangle_{A'B'}, \{\mathsf{M}'_x\}_{x\in X}, \{\mathsf{M}'_y\}_{y\in Y})$, denoted as $|\Psi\rangle_{AB} \overset{\rightarrow}{=}_\delta \mathsf{spec}$, if there is a local isometry $V_{AB} = V_A \otimes V_B : \mathcal{H}_A \otimes \mathcal{H}_B \to \mathcal{H}_{A'A''} \otimes \mathcal{H}_{B'B''}$ and a state $|\mathrm{junk}\rangle_{A''B''}$ such that

$$\|V_{AB}|\psi\rangle_{AB} - |\psi'\rangle_{A'B'} \otimes |\mathrm{junk}\rangle_{A''B''}\| \leq \delta \tag{2}$$

$$\left\|V_{AB}(\mathsf{M}_x \otimes \mathsf{M}_y|\psi\rangle)_{AB} - (\mathsf{M}'_x \otimes \mathsf{M}'_y \otimes |\psi'\rangle)_{A'B'} \otimes |\mathrm{junk}\rangle_{A''B''}\right\| \leq \delta \tag{3}$$

5.2 2-FAST functionality

We should point out the differences with previous definitions of adversarial self testing in the literature and argue why ours is appropiate in light of these previous definitions.

Definition 17. *We say a quantum channel $\mathcal{F}_{\text{2-FAST}}$ is a 2-FAST functionality if for some n, some specification* spec *associated to a non-local game G and some threshold τ, $\mathcal{F}_{\text{2-FAST}}$ is of the form*

$$\mathcal{F}_{\text{2-FAST}} : \mathsf{D}(\mathcal{H}_{A_1,\cdots,A_n} \otimes \mathcal{H}_{B_1,\cdots,B_n}) \to \mathsf{D}(\mathcal{H}_A^{out} \otimes \mathcal{H}_B^{out} \otimes S^{out}),$$

and given an input of the form $|\Psi\rangle_{\square_{A_1,\cdots,A_n,B_1,\cdots,B_n}}$, the functionality picks a random $j \in \{1, \cdots, n\}$ and outputs the register $\square_{A_j,B_j}$ together with a classical bitstring s, such that if $s =$ `abort` or $s = \perp$ then there's no condition on the output quantum state and if $s \neq$ `abort` or $s \neq \perp$ then $|\Psi\rangle_{\square_{A_j,B_j}} \overset{\rightarrow}{=}_{\delta(n,\tau)}$ spec.

Remark 18. Note that this defines a family of functionalities for 2-FAST, rather than a single functionality. The fact that the output is defined up to local isometries implies that any member of the family is a 2-FAST functionality.

The protocol $\Pi_{\mathcal{F}_{\text{2-FAST}}}$ in cite Protocol. Actually maybe I'll just add figure. can be shown to realise the 2-FAST functionality $\mathcal{F}_{\text{2-FAST}}$.

Lemma 19. *The protocol $\Pi_{\mathcal{F}_{\text{2-FAST}}}$ implements a 2-FAST functionality $\mathcal{F}_{\text{2-FAST}}$ with $\delta(n, \tau, \text{spec}) =$ give the expression for δ.*

Protocol 2 $\Pi_{\mathcal{F}_{\text{2-FAST}}}$

Global inputs. A reference specification $(|\psi'\rangle_{A'B'}, \{M'_x\}_x, \{M'_y\}_y)$ associated to a non-local game $(\mathcal{X}, \mathcal{Y}, \mathcal{A}, \mathcal{B}, \text{pred})$. A natural number n. A real number τ.
Inputs. A receives $\square_{A_1,\cdots,A_n}$, B receives $\square_{B_1,\cdots,B_n}$.

The protocol:

1. Each party sends their input directly to the trusted party.
2. The trusted party picks $i \leftarrow [n]$ and sets $J = [n] \setminus i$.
3. The trusted party runs the following loop. For each $j \in J$,
   (a) $(x, y) \leftarrow \mathcal{X} \times \mathcal{Y}$
   (b) $(a_j, b_j) \leftarrow \text{MeasureAB}_{xy}(\square_{A_j}, \square_{B_j})$
   (c) $v_j \leftarrow \text{pred}(a_j, b_j, x, y)$
4. Trusted party computes $v = \frac{1}{n-1}\sum_j v_j$. If $v \geq \tau$, the trusted party outputs $(\square_{A_i}, \{(a_j, b_j, x_j, y_j, j)\}_{j\in J})$ to party A and $(\square_{B_i}, \{(a_j, b_j, x_j, y_j, j)\}_{j\in J})$ to party B. Else the trusted party sends $\perp$ to both parties.

Although $\mathcal{F}_{\text{2-FAST}}$ captures what would be expected from a secure functionality for adversarial self-testing, we will in fact consider a slightly weaker functionality $\tilde{\mathcal{F}}_{\text{2-FAST}}$ together with a protocol $\Pi_{\tilde{\mathcal{F}}_{\text{2-FAST}}}$ for $\tilde{\mathcal{F}}_{\text{2-FAST}}$ given in Figure 2. This weakened version still gives a secure self-testing procedure with the difference that each party learns the index corresponding to the output and also learns the measurement setting for their own device. It's unclear whether the strong version of 2-FAST can be realised with the assumptions we use in our work or what extra assumptions would be needed to achieve it.

Add example showing why 2FAST is hard to realise, probably in a later section

5.3 Impossibility of 2-FAST with information-theoretic security

5.4 $\mathcal{F}_{\text{NL}}$ functionality

In realising the functionality $\tilde{\mathcal{F}}_{\text{2-FAST}}$, one is led naturally to a functionality testing the predicate of non-local games. In this section we introduce $\mathcal{F}_{\text{NL}}$, $\tilde{\mathcal{F}}_{\text{NL}}$ and $\mathcal{F}_{\text{NL}'}$ as ideal functionalities testing such non-local games.

First, we introduce $\mathcal{F}_{\text{NL}}$, and its ideal protocol as $\Pi_{\mathcal{F}_{\text{NL}}}$. The diagram describing the functionality $\mathcal{F}_{\text{NL}}$ is given in Figure 3.

Protocol 3 Ideal protocol $\Pi_{\mathcal{F}_{\mathsf{NL}}}$

Global inputs. A non-local game $(\mathcal{X}, \mathcal{Y}, \mathcal{A}, \mathcal{B}, \mathcal{D}, \mathsf{pred})$.

Inputs. A receives $\square_A$, B receives $\square_B$.

The protocol:

1. Each party sends their input directly to the trusted party.
2. Trusted party picks $(x, y) \xleftarrow{\mathcal{D}} \mathcal{X} \times \mathcal{Y}$.
3. Trusted party measures $(a, b) \leftarrow \mathsf{MeasureAB}_{xy}(\square_A, \square_B)$
4. Trusted party sends $abxy$ to party A and waits for response.
5. If Party A responds with `abort`, the trusted party outputs `abort` to B. If party A responds with any other message, then the trusted party outputs $abxy$ to party B.

The functionalities $\mathcal{F}_{\text{2-FAST}}$ and $\mathcal{F}_{\mathsf{NL}}$ are natural as definitions of functionalities for the purpose of fully adversarial self testing protocols. Proving the simulation security of some protocol with respect to $\mathcal{F}_{\text{2-FAST}}$ guarantees that no matter the behaviour of the adversary, if the procedure is a success then we will obtain a state which is approximately locally isometric to the reference state.

Just as in the case of $\mathcal{F}_{\text{2-FAST}}$, we will consider a weak version of $\mathcal{F}_{\mathsf{NL}}$ which we denote as $\tilde{\mathcal{F}}_{\mathsf{NL}}$. We will use this functionality to construct a protocol which realises $\tilde{\mathcal{F}}_{\text{2-FAST}}$. The functionality $\tilde{\mathcal{F}}_{\mathsf{NL}}$ has two phases, one in which the random measurement inputs is provided to each party and a second one where the quantum device states are given as input to the trusted party and the measurement results are reported. This means that step 2 of the protocol $\Pi_{\mathcal{F}_{\mathsf{NL}}}$ is modified by sending the sampled x, y to each corresponding party and then continuing execution of the protocol. This functionality is given Figure 4. Note that $\tilde{\mathcal{F}}_{\mathsf{NL}}$ reports the measurement input settings which are uncorrelated with the the states to be measured.

6 Protocols for 2-FAST and $\tilde{\mathcal{F}}_{\mathsf{NL}}$

6.1 Protocol for $\tilde{\mathcal{F}}_{\mathsf{NL}}$ from commitments

In order to implement $\mathcal{F}_{\mathsf{NL}}$, we define the commit-and-prove classical functionality $\mathcal{F}_{\mathsf{CP}}$ from [CLOS02]. This is a reactive functionality with two phases and runs with two parties, a committer C and a receiver R. The ideal protocol $\Pi_{\mathcal{F}_{NL}}$ is given as follows:

- In the "commit" phase, the functionality receives a classical message $w \in \{0,1\}^k$ from the committer C and sends a message acknowledging the received commitment to the receiver. The committed message is saved as an internal state in the functionality.
- In the "prove" phase, receives again a message $v \in \{0,1\}^k$ and the functionality checks whether $v = w$. If this is the case, then the functionality sends to R the bitstring w, otherwise it sends $\perp$.

The functionality $\mathcal{F}_{\mathsf{CP}}$ is given in Figure 5 and is defined as follows.

Definition 20. *The commit-and-prove functionality is defined by the commit functionality implementing the commit phase and the prove functionality implementing the prove phase. Functionalities keep in memory the classical string provided in the commit phase. The commit phase is given by the functionality*

$$\mathcal{F}_{commit}(s, *) = (received, \mathtt{ack}), \tag{4}$$

$$\mathcal{F}_{commit}(*, s) = (\mathtt{ack}, received). \tag{5}$$

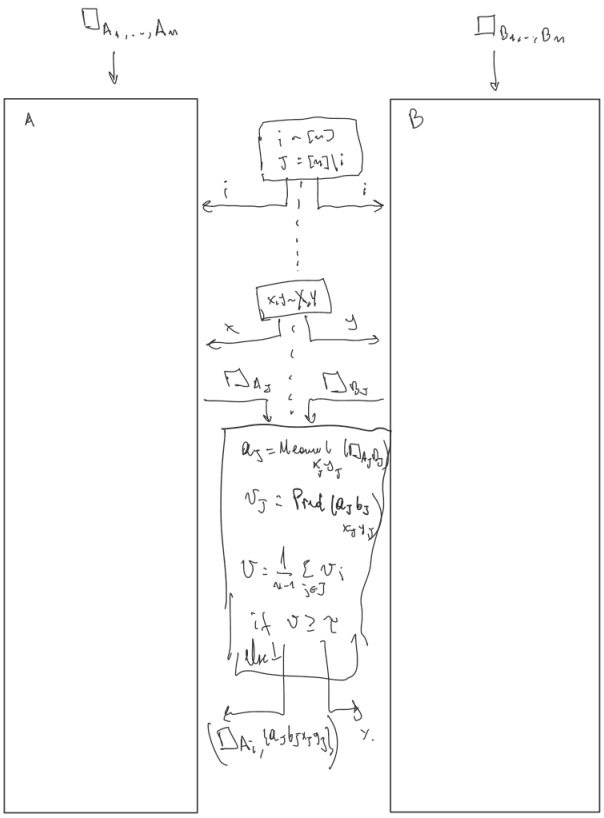


Figure 2: $\tilde{\mathcal{F}}_{\text{2-FAST}}$: Ideal functionality for executing a weak version of 2-FAST with device $(\square_A, \square_B)$.

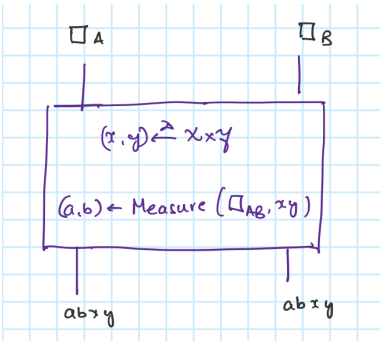


Figure 3: $\mathcal{F}_{\text{NL}}$: Ideal functionality for executing a non-local game with device $(\square_A, \square_B)$.

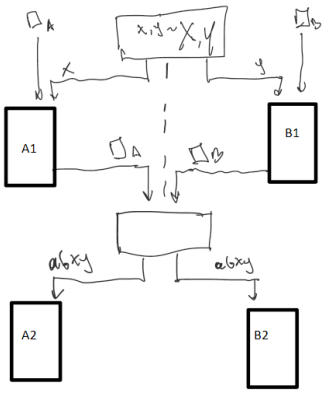


Figure 4: $\tilde{\mathcal{F}}_{\mathsf{NL}}$: Ideal functionality for executing a non-local game with device $(\square_A, \square_B)$. Weaker version

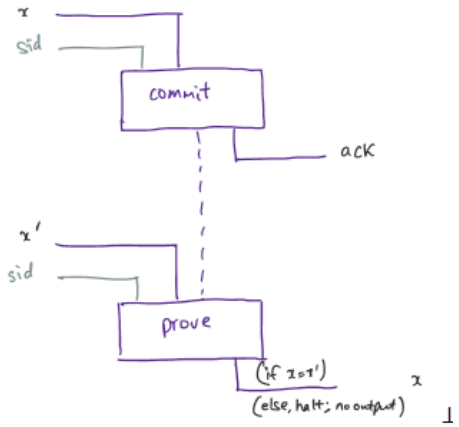


Figure 5: Commit-and-prove functionality. In the commit phase an input x is commited and the other party receives an acknowledgment. In the prove phase the, the commiting party shows the commited input to the receiving party.

Where $\mathcal{F}_{commit}$ has the input of the first party as the first input and that of the second party as second input. Once a party has commited to a string s, it receives a message $received$ and the second party receives `ack` *confirming the commitment. The prove phase is given by*

$$\mathcal{F}_{prove}(s', *) = \begin{cases} (*, s), & \text{if } s' = s \\ (*, \perp), & \text{otherwise} \end{cases} \tag{6}$$

$$\mathcal{F}_{prove}(*, s') = \begin{cases} (s, *), & \text{if } s' = s \\ (\perp, *), & \text{otherwise} \end{cases} \tag{7}$$

To QSA-realise the ideal protocol $\Pi_{\mathcal{F}_{\mathsf{NL}}}$ we define a $\mathcal{F}_{\mathsf{CP}}$-hybrid protocol $\Pi^{\mathcal{F}_{\mathsf{CP}}}$ in Figure 9. We use this protocol to QSA-realise protocol $\Pi_{\mathcal{F}_{\mathsf{NL}'}}$ defined in Figure 6.

Lemma 21. *Protocol $\Pi^{\mathcal{F}_{\mathsf{CP}}}$ QSA-realises $\Pi_{\mathcal{F}_{\mathsf{NL}'}}$.*

Proof. Let $\mathcal{A}$ be an adversary for $\Pi^{\mathcal{F}_{\mathsf{CP}}}$. Assume $\mathcal{A}$ corrupts party B. The case where party A is corrupted instead is similar except for the fact that there's no abort option before the other party receives the full output. We construct an adversary $\mathcal{S}$ for protocol $\Pi_{\mathcal{F}_{\mathsf{NL}'}}$ such that $M_{\Pi^{\mathcal{F}_{\mathsf{CP}}},\mathcal{A}} \approx_{\mathsf{qc}} M_{\Pi_{\mathcal{F}_{\mathsf{NL}'}},\mathcal{S}}$. To simplify notation, we define the QTM's $M_{\mathcal{A}} := M_{\Pi^{\mathcal{F}_{\mathsf{CP}}},\mathcal{A}}$ and $M_{\mathcal{S}} := M_{\Pi_{\mathcal{F}_{\mathsf{NL}'}},\mathcal{S}}$.

Any adversary $\mathcal{A}$ can be decomposed in 3 sequential protocols B_1^*, B_2^* and B_3^* which we use to construct $\mathcal{S}$. These protocols replace the honest protocols B_1, B_2 and B_3 in Figure 9. The adversary $\mathcal{S}$ is defined as follows:

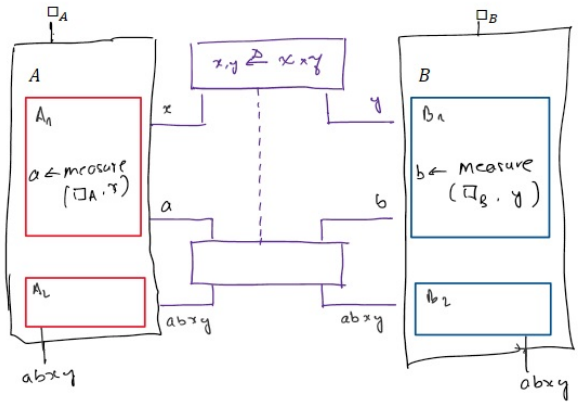


Figure 6: Protocol $\Pi_{\mathcal{F}_{\mathsf{NL}}'}$ for an input $(\square_A, \square_B)$. The execution of A and B can be decomposed into two parts.

1. $\mathcal{S}$ begins by receiving y from the trusted party, then it generates an `ack` message which is fed to B_1^* which gives x_B, y_B as an output together with any information about the state of the computation which is given as input to B_2^*. Then, $\mathcal{S}$ sets $y_A = y \oplus y_B$ and inputs y_A together with all other outputs from B_1^* to B_2^*. From B_2^*, output b is obtained together with any other output. Then, adversary $\mathcal{S}$ sends b to the trusted party and receives back from it $abxy$.

2. $\mathcal{S}$ then computes $x_A = x \otimes x_B$ and inputs x_A, a to B_3^*, if B_3^* outputs `abort` then $\mathcal{S}$ aborts as well otherwise it outputs what B_3^* outputs.

A diagramatic description of $\mathcal{S}$ is given in Figure 7. For a given $\mathcal{A}$, we show that both adversaries give the same outputs when given the same inputs and are therefore indistinguishable. Concretely, we show that the quantum channels defined by $M_{\mathcal{A}}$ and $M_{\mathcal{S}}$ are the same. Note that the commit-and-prove functionalities are assumed to be implemented by a trusted party and therefore the malicious party cannot break the security of the commitments.

First, we construct the quantum channel $\Phi_{\mathcal{A}}$ defined by the protocol $M_{\mathcal{A}}$. Let $\rho_{AB} \in \square_A \otimes \square_B$ be the input state to the protocol. We denote as T the register containing the messages sent by the parties and stored variables. It will be clear from the definition of the protocol which variables and messages the parties can access. At the beginning of the protocol the input is ρ_{AB}. The adversary $\mathcal{A}$ acts with some channel $\mathcal{E}_0^B$ over register $\square_B$ and outputs a register R. Hence, following this procedure the input is $\sigma_{AR} := \mathcal{E}_0^B(\rho_{AB})$.

After running A_1, the state is

$$\xrightarrow{A_1} \frac{1}{|X||Y|} \sum_{x_A \in X, y_A \in Y} |x_A y_A\rangle\langle x_A y_A|_T \otimes \sigma_{AR}. \tag{8}$$

We omit the `ack` messages for compactness. Next, B_1^* applies a quantum channel and measures to obtain classical variables x_B, y_B. Thus we have

$$\xrightarrow{B_1^*} \frac{1}{|X||Y|} \sum_{\substack{x_A y_A \\ x_B y_B}} |x_A y_A x_B y_B\rangle\langle x_A y_A x_B y_B|_T \otimes \left[\left(\mathbb{I}_A \otimes \mathcal{E}^R_{x_B y_B}\right) \sigma_{AR}\right]. \tag{9}$$

As the notation indicates, the channel $\mathcal{E}^R_{x_B y_B}$ acts only on on the register R and in general depends on the value of the variables x_B and y_B. Then, A_2 sets a variable $x = x_A \oplus x_B$ and measures the quantum device according to the measurement description in $\square_A$ given by $\mathsf{M}_x = \sum_a \Pi_{a|x}$.

$$\xrightarrow{A_2} \frac{1}{|X||Y|} \sum_{\substack{x_A y_A \\ x_B y_B \\ a}} |x_A y_A x_B y_B x a\rangle\langle x_A y_A x_B y_B x a|_T \otimes \left[\left(\mathcal{E}^A_{a|x} \otimes \mathcal{E}^R_{x_B y_B}\right) \sigma_{AR}\right]. \tag{10}$$

Where $\mathcal{E}^A_{a|x}(\rho) = \Pi_{a|x}\rho\Pi_{a|x}$. After B^*_2 and B^*_3 the state of the system is as follows.

$$\xrightarrow{B^*_2,B^*_3} \frac{1}{|X||Y|}\sum_{\substack{x_A y_A\\ x_B y_B\\ ab}} |x_A y_A x_B y_B xyab\rangle\langle x_A y_A x_B y_B xyab|_T \otimes \left[\left(\mathcal{E}^A_{a|x}\otimes(\mathcal{E}^R\circ\mathcal{E}^R_{b|y_A y_B}\circ\mathcal{E}^R_{x_B y_B})\right)\sigma_{AR}\right]. \tag{11}$$

The channel $\mathcal{E}^R_{b|y_A y_B}$ depends on variables y_A, y_B and the measurement result b but note that it could also depend on x_B and any variables previously seen by the adversary. The channel $\mathcal{E}^R$ depends on previous variables stored x_A, x_B, y_A, y_B, b, a since it has already received them from the trusted party. This defines the channel $\Phi_{\mathcal{A}}$ for the most general strategy followed by $\mathcal{A}$. We can assume that in the final output party A outputs just the variables $abxy$ and $\mathcal{A}$ outputs register R.

Now we show that $M_{\mathcal{S}}$ defines a channel $\Phi_{\mathcal{S}}$ and that $\Phi_{\mathcal{S}} = \Phi_{\mathcal{A}}$. From Figure 7 we see that the input is

$$\frac{1}{|X||Y|}\sum_{x,y}|xy\rangle\langle xy|\otimes\rho_{AB} \tag{12}$$

First, protocol A_1 measures the observable of the quantum device as in the previous case and $\mathcal{S}$ applies the channel $\mathcal{E}^{BB'}_0$ and runs B^*_1 obtaining

$$\xrightarrow{A_1,B^*_1} \frac{1}{|X||Y|}\sum_{\substack{x,y\\ a,x_B,y_B}} |xyax_B y_B y_A\rangle\langle xyax_B y_B y_A|_T \otimes \left[\left(\mathcal{E}^A_{a|x}\otimes\mathcal{E}^R_{x_B y_B}\right)\sigma_{AR}\right]. \tag{13}$$

Where $y_A = y \oplus y_B$. The protocol B^*_2 is run with y_A and register R as input and then B^*_3, giving

$$\xrightarrow{B^*_2,B^*_3} \frac{1}{|X||Y|}\sum_{\substack{x,y\\ x_B,y_B\\ a,b}} |xyabx_B y_B y_A\rangle\langle xyabx_B y_B y_A|_T \otimes \left[\left(\mathcal{E}^A_{a|x}\otimes\left(\mathcal{E}^R\circ\mathcal{E}^R_{b|y_A y_B}\circ\mathcal{E}^R_{x_B y_B}\right)\right)\sigma_{AR}\right]. \tag{14}$$

Note that compared to Equation (11), the sum is over the variables x, y instead of x_A, x_B. This is simply a change of variables in the sum and hence $\Phi_{\mathcal{S}} = \Phi_{\mathcal{A}}$.

□

Lemma 22. *Protocol $\Pi_{\mathcal{F}_{\mathsf{NL}'}}$ QSA-realises $\Pi_{\tilde{\mathcal{F}}_{\mathsf{NL}}}$.*

Proof. We follow the same idea in the proof of Lemma 21. For each adversary $\mathcal{A}$ for $\Pi_{\mathcal{F}_{\mathsf{NL}'}}$ we construct an adversary simulator $\mathcal{S}$ such that $M_{\Pi_{\mathcal{F}_{\mathsf{NL}'}},\mathcal{A}}$ and $M_{\Pi_{\tilde{\mathcal{F}}_{\mathsf{NL}}},\mathcal{S}}$ are indisinguishable. In fact, the quantum channels $\Phi_{\mathcal{A}}$ and $\Phi_{\mathcal{S}}$ defined by these QTMs are the same. Note that the parties can only implement efficient (quantum poly-time) protocols. We assume that $\mathcal{A}$ and $\mathcal{S}$ corrupt party B without loss of generality.

First, we describe the quantum channel induced by $M_{\mathcal{A}}$ for a general adversary $\mathcal{A}$. Given an adversary $\mathcal{A}$ for $\Pi_{\mathcal{F}_{\mathsf{NL}'}}$, decompose the procedure of the corrupt party into the procedures B^*_1 and B^*_2 and that of the honest party into A_1, A_2 as in Figure 6. The input is given by $\frac{1}{|X||Y|}\sum_{x,y}|xy\rangle\langle xy|\otimes\rho_{AB}$, and the state of the system after A_1 is

$$\xrightarrow{A_1} \frac{1}{|X||Y|}\sum_{x,y,a}|xya\rangle\langle xya|\otimes\left[\left(\mathcal{E}^A_{a|x}\otimes\mathbb{I}_B\right)\rho_{AB}\right]. \tag{15}$$

Channel $\mathcal{E}^A_{a|x}$ implements the quantum device measurements, this includes measuring in the computational basis the register S_A to obtain a description of the measurements and then implementing them.

Procedure B^*_1 can be seen as a unitary channel $\mathcal{U}_1$ applied on input y, register $\square_B = \mathcal{H}_B \otimes S_B$ and an ancilla register. A part of the output is assumed to be measured in the computational basis which gives output b, and the non-measured register is passed onto B^*_2. We denote the register that has not been measured as R. The procedure outlined defines a channel $\mathcal{E}^B_{b|y} : \mathsf{L}(\mathcal{H}_B \otimes S_B) \to \mathsf{L}(R)$. Note that the output b is messaged to the trusted party. B^*_1 could potentially apply an operation which depends on b after the measurement but if that's the case then we assume B^*_2 applies it instead. Protocol B^*_2 receives as inputs the register R, the output of the trusted party $abxy$ and some ancilla register, then applies the unitary channel $\mathcal{U}_2$ and outputs the register R_{out} giving a channel

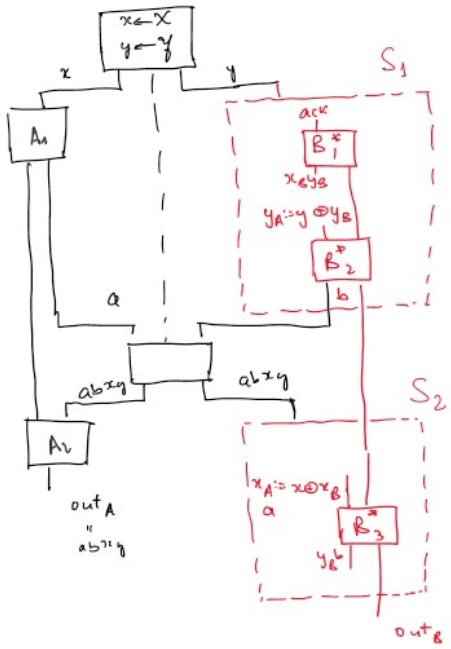


Figure 7: Adversary $\mathcal{S}$ (in red) corrupting party B for protocol $\Pi_{\mathcal{F}_{\mathsf{NL}}'}$ with input $(\square_A, \square_B)$. The procedure for $\mathcal{S}$ is decomposed into $\mathcal{S}_1$ and $\mathcal{S}_2$

$\mathcal{E}^R_{abxy} : \mathsf{L}(R_2) \to \mathsf{L}(R_{out})$. A diagramatic description of these channels in terms of the unitary channels is given in Figure 8.

Therefore, we have that the channel $\Phi_{\mathcal{A}}$ associated to $M_{\mathcal{A}}$ is given by

$$\Phi_{\mathcal{A}}(\rho_{AB}) = \frac{1}{|X||Y|} \sum_{x,y,a,b} |xyab\rangle\langle xyab| \otimes \left[\left(\mathcal{E}^A_{a|x} \otimes \mathcal{E}^R_{abxy} \mathcal{E}^B_{b|y} \right) \rho_{AB} \right]. \tag{16}$$

Now we proceed to describe simulator $\mathcal{S}$.

1. When $\mathcal{S}$ receives y and $\square_B$, create an ancilla register and apply $\mathcal{U}_1$. Two output registers are obtained, R' which is supposed to be measured to obtain b, and register R.
2. Adjoin to R' a register describing the measurement performed by $\mathcal{A}$, denoted as $\tilde{S}_B$. Send both $\square'_B = R' \otimes \tilde{S}_B$ to the trusted party.
3. After receiving $abxy$ from the trusted party and R from the previous step, run B_2^* and get R_{out}.

Clearly, these operations define a channel $\Phi_{\mathcal{S}}$ such that $\Phi_{\mathcal{S}} = \Phi_{\mathcal{A}}$. In step 1, $\mathcal{S}$ simply follows the same protocol as $\mathcal{A}$ untill before the measurement of b.

□

6.2 Protocol for 2-FAST from $\tilde{\mathcal{F}}_{\mathsf{NL}}$

It remains to show that $\Pi_{\tilde{\mathcal{F}}_{\text{2-FAST}}}$ can be QSA-realised by a protocol in the $\tilde{\mathcal{F}}_{\mathsf{NL}}$-hybrid model. We denote this protocol as $\Gamma^{\tilde{\mathcal{F}}_{\mathsf{NL}}}$. We give the definition of $\Gamma^{\tilde{\mathcal{F}}_{\mathsf{NL}}}$ in Figure 10.

Lemma 23. *$\Gamma^{\tilde{\mathcal{F}}_{\mathsf{NL}}}$ QSA-realises $\Pi_{\tilde{\mathcal{F}}_{\text{2-FAST}}}$.*

Proof. Let $\mathcal{A}$ be an adversary for $\Gamma^{\tilde{\mathcal{F}}_{\mathsf{NL}}}$ which without loss of generality corrupts party A. We can decompose the adversary into for protocols as in add figure. First, protocol A_1^* receives register $\square_A = \square_{A_1,\cdots,A_n}$ and applies a

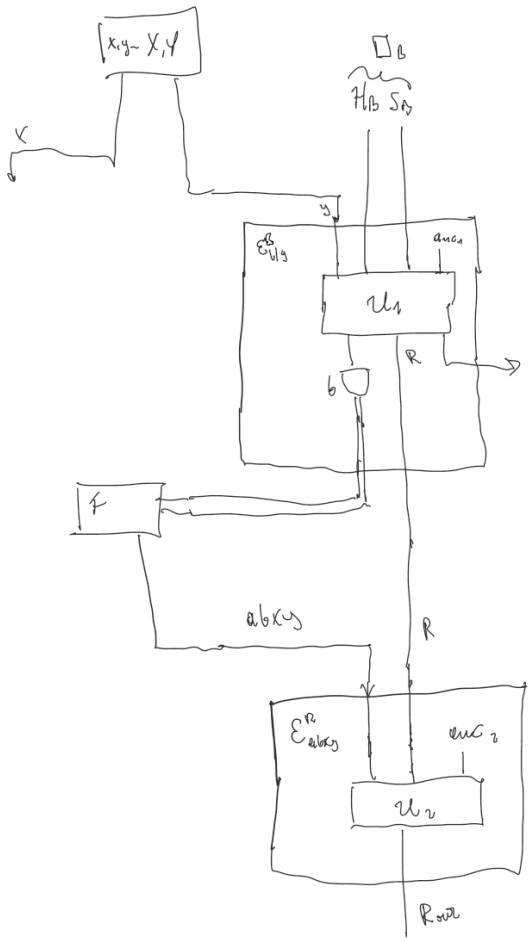


Figure 8: Most general strategy for adversary $\mathcal{A}$ during execution of $\Pi^{\mathcal{F}_{\mathrm{NL}'}}$. We omit the stratetgy of the honest party A. Need to get rid of registers which are just discarded

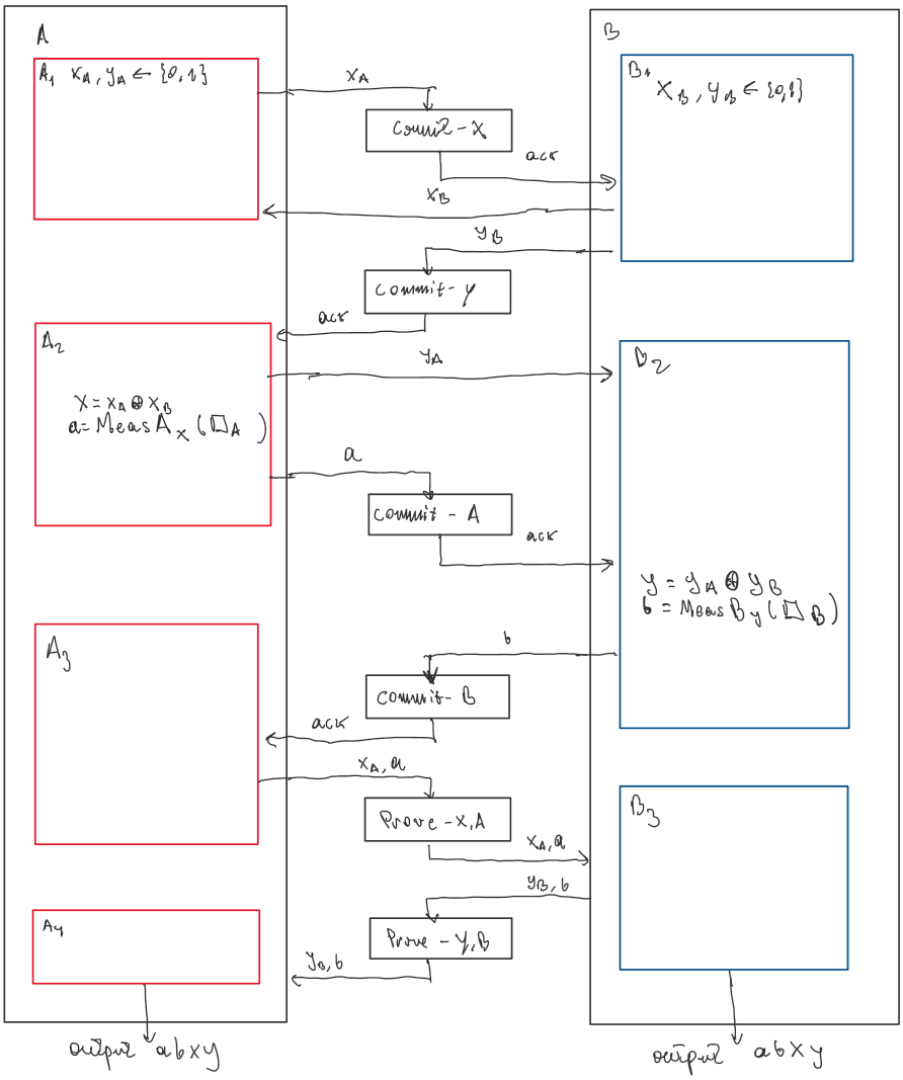


Figure 9: Protocol $\Pi^{\mathcal{F}_{\text{CP}}}$ for an input $(\square_A, \square_B)$.

quantum channel $\mathcal{E}^A : \mathsf{L}(\mathcal{H}_A \otimes S_A) \to \mathsf{L}(R^{(1)})$ to ρ_{AB} describing the state in $\square_{A,B}$. The output is a register $R^{(1)}$ which is passed to the next protocol A_2^* and an integer $i_A^{(1)}$ which is copied on a register T and passed to the trusted party. Thus, the resulting state is

$$\xrightarrow{A_1^*} \sum_{i_A^{(1)}} \left| i_A^{(1)} \right\rangle\!\left\langle i_A^{(1)} \right|_T \otimes \left(\mathcal{E}^A_{i_A^{(1)}} \otimes \mathbb{I}^B \right)(\rho_{AB}). \tag{17}$$

The honest party B acts by sending a commitment to the trusted party, which the adversary sees as an `ack` message. Next, protocol A_2^* is run, which applies a channel $\mathcal{E}^{R^{(1)}} : \mathsf{L}(R^{(1)}) \to \mathsf{L}(R^{(2)})$. The outputs are given by a register $R^{(2)}$ which is passed to the next protocol and an integer $i_A^{(2)}$ which is used to prove the previous commitment and is added to register T. Note that $i_A^{(1)} = i_A^{(2)}$ otherwise the proving phase of the commitment would fail.

$$\xrightarrow{A_2^*} \sum_{i_A^{(1)}} \left| i_A^{(1)} \right\rangle\!\left\langle i_A^{(1)} \right|_T \otimes \left(\mathcal{E}^{R^{(1)}} \circ \mathcal{E}^A_{i_A^{(1)}} \otimes \mathcal{E}^B \right)(\rho_{AB}). \tag{18}$$

The protocol A_3^* receives the register $R^{(2)}$ and the register T with an integer i_B generated randomly by party B. The protocol then applies $\mathcal{E}^{R^{(2)}} : \mathsf{L}(R^{(2)} \otimes T) \to \mathsf{L}(R^{(3)} \otimes \square_{A_{i^*}} \otimes \square A_{J^*})$ where i^* would correspond in the honest case to $i = i_A^{(1)} \oplus i_B$ and J^* would correspond to J.

$$\xrightarrow{A_3^*} \frac{1}{n} \sum_{i_A^{(1)}, i_B} \left| i_A^{(1)}, i_B \right\rangle\!\left\langle i_A^{(1)}, i_B \right|_T \otimes \left(\mathcal{E}^{R^{(2)}}_{i_B} \circ \mathcal{E}^{R^{(1)}} \circ \mathcal{E}^A_{i_A^{(1)}} \otimes \mathcal{E}^B \right)(\rho_{AB}). \tag{19}$$

The final protocol A_4^* receives register $R^{(3)}$, $\square_{A_{i^*}}$ and a register with the final output from the trusted party containing $\{a_j, b_j, x_j, y_j\}_{j \in J}$. For convenience, we write $x_J = (x_i)_{i \in J}$ and similarly for a_J, b_J and y_J. Moreover, let

$$\sigma = \left(\mathcal{E}^{R^{(2)}}_{i_B} \circ \mathcal{E}^{R^{(1)}} \circ \mathcal{E}^A_{i_A^{(1)}} \otimes \mathcal{E}^B \right)(\rho_{AB}). \tag{20}$$

Note that $\sigma \in \mathsf{D}(R^{(2)} \otimes \square_{A_{i^*}} \otimes \square_{A_{J^*}} \otimes \square_{B_J} \otimes \square_{B_i})$. After the trusted party runs its protocol we get the state

$$\xrightarrow{\text{Trusted party}} \frac{1}{n|X|^J|Y|^J} \sum_{i_A^{(1)}, i_B, x_J, y_J, a_J, b_J} \left| i_A^{(1)}, i_B, x_J, y_J, a_J, b_J \right\rangle\!\left\langle i_A^{(1)}, i_B, x_J, y_J, a_J, b_J \right|_T \otimes \left(\mathbb{I}^{R^{(2)} A_{i^*}} \otimes \mathcal{E}^{A_{J^*} B_J}_{x_J y_J a_J b_J} \otimes \mathbb{I}^{B_i} \right)(\sigma). \tag{21}$$

Finally, the protocol A_4^* is run and party B applies a last channel

$$\xrightarrow{A_4^*} \frac{1}{n|X|^J|Y|^J} \sum_{i_A^{(1)}, i_B, x_J, y_J, a_J, b_J} \left| i_A^{(1)}, i_B, x_J, y_J, a_J, b_J \right\rangle\!\left\langle i_A^{(1)}, i_B, x_J, y_J, a_J, b_J \right|_T \otimes \left(\mathcal{E}^{R^{(2)} A_{i^*}}_{x_J y_J a_J b_J} \otimes \mathcal{E}^{A_{J^*} B_J}_{x_J y_J a_J b_J} \otimes \mathcal{E}^{B_i} \right)(\sigma). \tag{22}$$

Now we construct an adversary $\mathcal{S}$ for $\Pi_{\tilde{\mathcal{F}}_{\text{2-FAST}}}$ from $\mathcal{A}$ such that the outputs are the same. To keep the proof short we will sketch out the proof but basically follows a similar reasoning for constructing a general adversary $\mathcal{A}$.

It's clear that after the two protocols run by the trusted party in $\Pi_{\tilde{\mathcal{F}}_{\text{2-FAST}}}$ the state of the whole system is

$$\frac{1}{n|X|^J|Y|^J} \sum_{i, x_J, y_J} |i, x_J, y_J\rangle\langle i, x_J, y_J|_T \otimes \left(\mathcal{E}^{R^{(2)}}_{i_B} \circ \mathcal{E}^{R^{(1)}} \circ \mathcal{E}^A_{i_A^{(1)}} \otimes \mathcal{E}^B \right)(\rho_{AB})$$

where adversary $\mathcal{S}$ has sampled randomly an integer $i_A^{(1)}$ and then set $i_B = i \oplus i_A^{(1)}$. Then it applies the same channels as $\mathcal{A}$. Party B also applies the corresponding channels. The last step of $\mathcal{S}$ simply consists in sending $\square_{A_{J^*}}$ in place of $\square_{A_J}$ and $\square_{A_{i^*}}$ in place of $\square_{A_i}$. A similar calculation as previous ones shows that the channel induced by $\mathcal{A}$ and $\mathcal{S}$ are the same.

□

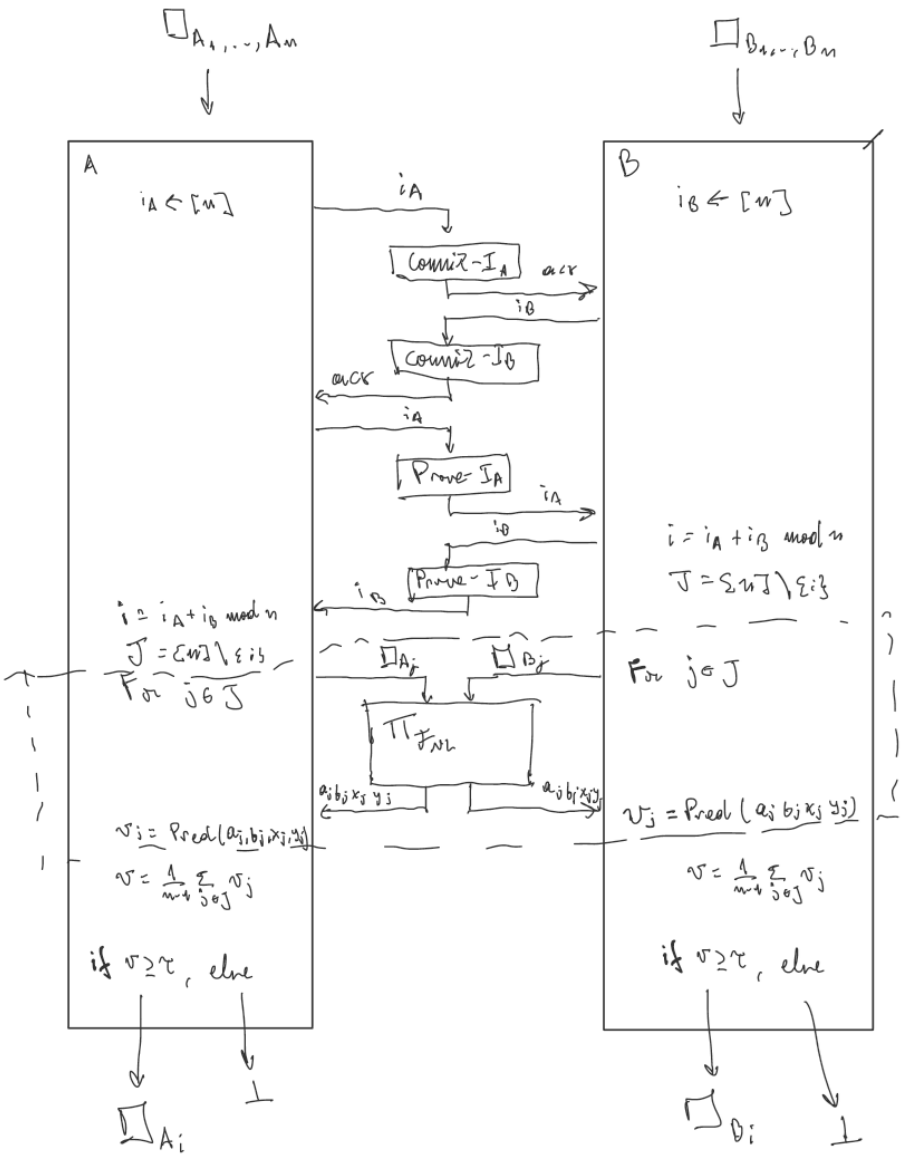


Figure 10: Protocol $\Gamma^{\mathcal{F}_{\text{NL}}}$. This protocol QSA-realises $\Pi_{\mathcal{F}_{\text{2-FAST}}}$ with input $(\square_{A_1,\cdots,A_n}, \square_{B_1,\cdots,B_n})$. Need to change the figure to "parallel" version. Another note to myself: with parallel version I mean that all the boxes are given as input to the functionality, so in fact there's only one run of the functionality.

7 Application: Coin flipping with everlasting security

Coin flipping is the following two-party computation problem: Alice and Bob do not trust each other and be exchanging messages (without involving any trusted third party) they wish to agree on a random bit. Ideally, one would like to show security without making any assumptions about the computationally abilities of the adversaries, i.e. to have information theoretic security. However, coin flipping in particular, is impossible to achieve in this setting. Even if quantum messages are exchanged. Under public-key like assumptions, it is well known that coin flipping can be achieved classically and its security can be extended against quantum adversaries as well. The discussion so far seems to suggest that quantum mechanics offers no advantage in solving the coin flipping problem.

One possibility is considering a potentially weaker primitive. Indeed, it turns out that if one weakens the notion of security of coin flipping and considers the so called *weak coin flipping problem*—where the players still want to share a random bit have known opposite preferences for the bit—then quantum mechanics does offer an advantage. One can show that in the information theoretic setting, no secure classical protocol for this problem exists while quantumly, protocol families approaching perfect security exist.

Another possibility is to strengthen the notion of security for coin flipping (instead of weakening it) and see if quantum mechanics helps achieve it. Since information theoretic security is impossible, one can consider *everlasting security*. This section gives formal evidence that quantum mechanics offers an advantage in constructing coin flipping protocols with everlasting security.

To proceed, we briefly discuss how everlasting security is defined. Consider two-party protocols that have an *offline* (setup) phase—which can be run before the parties know their inputs—and an *online* (execution) phase—which will typically depend on the party's inputs. Suppose that adversaries are computationally bounded in the offline phase but are allowed to be unbounded in the online phase. This models the situation where the underlying computational hardness assumption in the protocol is not broken during the setup phase, but gets broken later. Everlasting security requires the protocol to stay secure in this model.[4]

We study the problem of coin flipping with everlasting security by considering unconditional security in the following two models: (a) Shared randomness or the CRS[5], the classical setting and (b) Shared-EPR, the quantum setting. How do these models relate to everlasting security? The idea is simply to use computational boundedness of the adversaries in the offline phase to share randomness/EPR pairs among untrusted parties. More precisely, from commitments, one can create shared randomness and similarly, from 2-FAST (which in turn can also be achieved using the commitments[6], see Section 6), one can also create shared EPR states. To study everlasting coin-flipping, we therefore restrict to studying information theoretic security in the shared randomness/EPR model.[7][8]

Henceforth, security in this section is assumed to mean information theoretic security. Section **??** recalls coin flipping in the plain model (i.e. with no a priori shared resources among the parties), introduces the relevant security notion denoted by $\epsilon \in [0, 1/2]$ and termed *bias*. It then recalls the known result that secure coin flipping is impossible: Classically, no coin flipping protocol can have bias less than $1/2$ (worst possible), while quantumly, no coin flipping protocol can have bias less than $1/\sqrt{2} - 1/2$. Section **??** introduces the shared randomness setting together with the relevant security notion termed *predictability* (strengthening of *bias*; predictability is at least $1/2 + \epsilon$) which is a probability and so takes values in $[0, 1]$. The section then describes a very simple coin flipping protocol in this setting which has predictability $3/4$ and goes on to prove that no protocol can achieve predictability less than $3/4$. An ideal protocol should have predictability exactly $1/2$. Thus classically, secure coin flipping is impossible in this model. Section **??** extends the notion of predictability to the quantum setting and gives a protocol in the shared-EPR setting that achieves predictability lower than $3/4$, thereby establishing quantum advantage for this problem. Section 7.4 shows how *bias* and *predictability* are consequences of simulation based security definitions for coin flipping and conjectures existence of coin flipping protocols with UC-security in the EPR model.

[4] TODO: mention that coin flipping doesn't have any inputs and mention some impossibility results about everlasting security from Unruh's work

[5] with uniform distribution

[6] the commit-and-prove variant

[7] TODO: check; It could be that this model does not capture the impossibility in general; because it could be that in the classical case, one uses computational assumptions to do something other than have a shared string; I'll have to think a bit and argue whether or not this is the case. But let's first get this to work.

[8] Patronising?

7.1 Impossibility of coin flipping in the plain model

We start by defining a coin flipping protocol as follows.

Definition 24 (Coin flipping protocol (in the plain model))**.** *A coin flipping protocol (in the plain model) is specified by two interacting machines A, B that on input 1^λ, interact with each other and output bits (a, b) respectively, denoted by $(a, b) \leftarrow \langle A, B\rangle$, in time at most $f(\lambda)$ for some fixed function $f : \mathbb{N} \to \mathbb{N}$. For classical protocols, these machines are classical and for quantum protocols, these machines are quantum. There are no bounds on their computational abilities, i.e. there are no constraints on f.*

We now setup some notation.

Notation 25. Fix any coin flipping protocol in the plain model and the security parameter λ. Denote by

- $p_{ab} := \Pr[(a, b) \leftarrow \langle A, B\rangle]$,
  the probability that Alice and Bob output (a, b) when they follow the protocol honestly.

We say a coin flipping protocol is *correct*, if $p_{11} = p_{00} = 1/2$ and $p_{01} = p_{10} = 0$. Denote by

- $p_{a*} := \max_{B'} \sum_{b'} \Pr[(a, b') \leftarrow \langle A, B'\rangle]$,
  the maximum probability with which an unbounded malicious Bob forces an honest Alice to output a, and by
- $p_{*b} := \max_{A'} \sum_{a'} \Pr[(a', b) \leftarrow \langle A', B\rangle]$,
  the maximum probability with which an unbounded malicious Alice forces an honest Bob to output b.

The figure of merit for coin flipping protocols in this model, is taken to be the *bias* ϵ which is defined as

$$\epsilon := \max\{p_{0*}, p_{1*}, p_{*0}, p_{*1}\} - 1/2. \tag{23}$$

Observe that for correct protocols, $\epsilon \in [0, \frac{1}{2}]$. Kitaev proved that the following holds for any coin flipping protocol (in the plain model).

Lemma 26. *Let p_{ab}, p_{a*}, p_{*b} be as above for any classical coin flipping protocol. Then, it holds that*[9]

$$(1 - p_{a*})(1 - p_{*b}) \leq \sum_{a' \neq a, b' \neq b} p_{a'b'} = p_{\bar{a},\bar{b}}.$$

Proof. See TODO: refer to appendix □

Note that Lemma 26 in particular means that $(1 - p_{1*})(1 - p_{*0}) \leq p_{01} = 0$ for any correct coin flipping protocol and thus, either $p_{*0} = 1$ or $p_{1*} = 1$, i.e. at least one player is able to cheat maximally. This proves that $\epsilon = 1/2$ for any (correct) coin flipping protocol, in the plain model.[10]

Kitaev also showed that even in the quantum setting, perfect coin flipping is impossible, by proving the following statement.

Lemma 27. *Let p_{ab}, p_{a*}, p_{*b} be as above for any quantum coin flipping protocol. Then, it holds that*

$$p_{a*} p_{*b} \geq p_{ab}.$$

Proof. See TODO: refer to appendix □

This, in particular, means that for any correct quantum coin flipping protocol, the bias is at least $\left(\frac{1}{\sqrt{2}} - \frac{1}{2}\right)$, since $p_{1*}p_{*1} \geq 1/2$ and similarly $p_{0*}p_{*0} \geq 1/2$, thus at least one of them is lower bounded by $1/\sqrt{2}$ which means the bias is as claimed. We thus have the following.

Theorem 28 (Impossibility of perfect coin flipping in the plain model)**.** *There is a universal constant $c := 1/\sqrt{2} - 1/2$ such that all correct (classical or quantum) coin flipping protocols (in the plain model) have bias at least c, i.e. $\epsilon(\lambda) \geq c$, for all values of the security parameter λ.*

[9] We use $\bar{a}$ to mean $a \oplus 1$.

[10] There may be older proofs of this fact

7.2 Coin flipping in the shared randomness model

Let us now look at coin flipping in the shared randomness model.[11] For simplicity, in this model we assume that each party in the protocol has access to a shared (potentially infinitely long) string $s \in \{0,1\}^*$ that has been drawn from the uniform distribution.

Definition 29 (Classical coin flipping protocol (in the shared randomness model))**.** *Define a classical coin flipping protocol exactly like Definition 24 except that A and B, take two inputs this time: in addition to 1^λ, they take as input a string, $s \leftarrow \{0,1\}^{k(\lambda)}$ which is a uniformly sampled string of length $k(\lambda)$ for some fixed function $k : \mathbb{N} \to \mathbb{N}$.*

The dependence on the security parameter λ is left implicit. The dependence on s is written as A^s and B^s and[12]

$$\begin{aligned}\Pr[(a,b) \leftarrow \langle A, B\rangle] &:= \sum_{s' \in \{0,1\}^{k(\lambda)}} \Pr\left[(a,b) \leftarrow \left\langle A^{s'}, B^{s'}\right\rangle\right] \cdot \Pr[s = s'] \\ &= \langle \Pr\left[(a,b) \leftarrow \langle A^s, B^s\rangle\right]\rangle_s .\end{aligned}$$

We use p_{ab}, p_{a*}, p_{*b} as in Notation 25. To be more precise, we introduce the following conditional probabilities.

Notation 30. Fix any classical coin flipping protocol in the shared randomness model (as in Definition 29) and the security parameter λ. Denote by

- $p^s_{ab} := \Pr\left[(a,b) \leftarrow \langle A^s, B^s\rangle\right]$,
  the probability that Alice and Bob output (a,b), given the random string is s
- $p^s_{a*} := \max_{B'} \sum_{b'} \Pr\left[(a,b') \leftarrow \langle A^s, B'^s\rangle\right]$,
- $p^s_{*b} := \max_{A'} \sum_{a'} \Pr\left[(a',b) \leftarrow \langle A'^s, B^s\rangle\right]$

Clearly, $p_{ab} = \langle p^s_{ab}\rangle_s$, $p_{a*} = \langle p^s_{a*}\rangle_s$ and $p_{*b} = \langle p^s_{*b}\rangle_s$.

Bias. Denote Alice's and Bob's bias by

$$\epsilon_A := \max_b p_{*b} - \frac{1}{2}, \text{ and } \epsilon_B := \max_a p_{a*} - \frac{1}{2}$$

respectively. Bias $\epsilon := \max\{\epsilon_A, \epsilon_B\}$ then coincides with Equation (23).

Predictability bias. Denote Alice's predictability bias given s, by

$$\varepsilon^s_A := \max_b p^s_{*b} - \frac{1}{2}, \text{ and by } \varepsilon_A := \langle \varepsilon^s_A\rangle_s$$

her predictability bias. Similarly, Bob's predictability bias given s is

$$\varepsilon^s_B := \max_a p^s_{a*} - \frac{1}{2}, \text{ and } \varepsilon_B := \langle \varepsilon^s_B\rangle_s .$$

Predictability bias is then given by $\varepsilon := \max\{\varepsilon_A, \varepsilon_B\}$.

Observe that predictability bias $\varepsilon \in [0, \frac{1}{2}]$ for correct protocols. Further, predictability bias upper bounds bias.

Remark 31. Note that $\epsilon_A = \max_b \langle p^s_{*b}\rangle_s - \frac{1}{2}$ and $\varepsilon_A = \langle \max_b p^s_{*b}\rangle_s - \frac{1}{2}$. Thus, $\epsilon_A \le \varepsilon_A$ and similarly, $\epsilon_B \le \varepsilon_B$.

To see why predictability bias is relevant, consider the following trivial coin flipping protocols in the shared randomness model.

Algorithm 32 (The dummy CF-CRS protocol)**.** *The dummy protocol specifies $k = 1$ (the length of s to be one bit) and specifies A^s and B^s to be dummy machines that simply output s, i.e. $\langle A^s, B^s\rangle$ outputs (s,s).*

Clearly, this protocol has bias $\epsilon = 0$ (most secure in terms of bias) because the bit s is independently (of the malicious party) chosen to be uniformly distributed. However, given s, a malicious party can trivially predict the output of the other party—it is exactly s. Predictability bias, therefore, must be $\varepsilon = 1/2$ (least secure in terms of predictability). Indeed, this also follows from the definition by simply computing the various quantities (see Table 1).

Let us now consider a protocol that has non-trivial ε.

[11] This model is more commonly referred to as common reference string model.

[12] In the last equality, we used the notation $\langle \mathsf{Expr}(s)\rangle_s := \sum_{s' \in S} \mathsf{Expr}(s') \Pr(s' = s)$ for some random variable taking values in S.

Algorithm 33 (The s-flips CF-CRS protocol)**.** *The s-flip protocol specifies $k = 1$, and specifies A^s and B^s as follows:*

- *When $s = 0$, Alice flips:*
  - *A^0 samples a random bit $r \leftarrow \{0,1\}$ and sends it to B^0.*
  - *A^0 outputs r and B^0 outputs whatever it receives.*
- *When $s = 1$, Bob flips:*
  - *B^1 samples a random bit $r \leftarrow \{0,1\}$ and sends it to A^1.*
  - *B^1 outputs r and A^1 outputs whatever it receives.*

In this case, from Table 1, it follows that $\varepsilon_A = \varepsilon_B = 1/4$. Can one construct a more clever protocol that does even better? No, it turns out that this is the best possible classical protocol in this model.

Theorem 34 (Impossibility of perfect classical coin flipping in the shared randomness model)**.** *There is a universal constant c such that all correct coin flipping protocols (in the shared randomness model) have predictability bias at least $\varepsilon(\lambda) \geq c$. For classical protocols, $c := 1/4$ while for quantum protocols $c := \frac{1}{2\sqrt{2}} - 1/4 \approx 0.35355 - 0.25 \approx 0.10355$.*

Proof. Restrict to correct protocols. Let $\eta_A^s := \max_b p_{*b}^s$ and $\eta_B^s := \max_b p_{a*}^s$. Note that $\langle \eta_A^s \rangle_s = \varepsilon_A + \frac{1}{2}$ and similarly $\langle \eta_B^s \rangle = \varepsilon_B + \frac{1}{2}$. By correctness, clearly, η_A^s and η_B^s are both at least $1/2$.

We start with classical protocols and observe the following.

Claim 35. For classical protocols, either $\eta_A^s = 1$ or $\eta_B^s = 1$.

Proof of Claim 35. To see this, recall that one can apply Lemma 26 for each s to conclude $(1 - p_{a*}^s)(1 - p_{*b}^s) \leq p_{\bar{a}\bar{b}}^s$ for all $a, b \in \{0,1\}$. For correct protocols, $p_{01}^s = p_{10}^s = 0$ and so both (1) and (2) hold where (1) $p_{1*} = 1$ or $p_{*0} = 1$ and (2) $p_{0*} = 1$ or $p_{*1} = 1$. This implies that either $\eta_A^s = 1$ or $\eta_B^s = 1$ as claimed.[13] □

Let the subset $S_A \subseteq \{0,1\}^k$ be such that for $s \in S_A$, it holds that $\eta_A^s = 1$ and $1 > \eta_A^s \geq 1/2$ otherwise. Suppose that S_A contains at least half the strings of length k, i.e. $|S_A| \geq 2^k/2$. Clearly, then $\eta_A \geq \frac{1}{2} \cdot 1 + \frac{1}{2} \cdot \frac{1}{2} \geq \frac{3}{4}$ and therefore, in this case, $\varepsilon_A \geq \frac{1}{4}$. If S_A is smaller than half the strings of length k, i.e. $|S_A| < 2^k/2$, then one can run the same argument (using Claim 35) but for B with $S_B = \{0,1\}^k \backslash S_A$. It follows that $\eta_B \geq \frac{1}{2} \cdot 1 + \frac{1}{2} \cdot \frac{1}{2} \geq \frac{3}{4}$ and so $\varepsilon_B \geq 1/4$ in that case. Since $\varepsilon = \max\{\varepsilon_A, \varepsilon_B\}$, it holds that $\varepsilon \geq 1/4$, completing the proof that $c = 1/4$ for classical protocols.

For quantum protocols, the analogue of Claim 35 is as follows.

Claim 36. For quantum protocols, either $\eta_A^s \geq 1/\sqrt{2}$ or $\eta_B^s \geq 1/\sqrt{2}$.

Proof of Claim 36. Proceed as above except that we use Lemma 27 for each s to conclude $p_{a*}^s p_{*b}^s \geq p_{ab}^s$ which for correct protocols yields both (1) and (2) where (1) $p_{1*}^s \geq 1/\sqrt{2}$ or $p_{*0}^s \geq 1/\sqrt{2}$ and (2) $p_{0*}^s \geq 1/\sqrt{2}$ or $p_{*1}^s \geq 1/\sqrt{2}$. □

And then in that case, proceeding analogously, ε would be at least $c = 1/2(1/\sqrt{2}+1/2) - 1/2 = 1/(2\sqrt{2}) - 1/4 \approx 0.10355$, completing the proof. □

The bound for the quantum case may not be tight. For now, since anyway our focus is on near-perfect device independent protocols, we leave open the question of an optimal (in terms of predictability bias) quantum protocol for coin flipping in the shared randomness model. For completeness, we give a protocol in the quantum + shared randomness model which achieves a bias predictability bias lower than that in the classical shared randomness model. We define the Basis-Commit protocol in Figure 11 which we will show achieves a predictability bias of $0.67677 - 0.5 =\approx 0.17677$. The protocol is similar to that of [Amb04] but achieves a lower bias by using the common random bit to symmetrize the protocol.

Lemma 37. *The Basis-Commit protocol in the quantum + CRS model achieves a predictability bias $\varepsilon \leq \frac{1}{2}(\cos^2(\frac{\pi}{8}) + \frac{1}{2} + \mathsf{negl}(\lambda)) - \frac{1}{2} \approx 0.17677$*

[13] In fact, even if only (1) was true, this would follow.

Proof. Since the Commitment-based protocol is symmetrized with respect to the role of Alice and Bob, $\varepsilon = \varepsilon_A = \varepsilon_B$. Therefore, we compute a bound for ε_A. By definition we have that $\varepsilon_A = \langle \max_b p^s_{*b} \rangle_s = \frac{1}{2}\max_b p^{s=0}_{*b} + \frac{1}{2}\max_b p^{s=1}_{*b}$.

Case $s = 1$: In this case, the best strategy for Alice involves measuring the states $H^x |a\rangle$ received from Bob and try to guess the parity of $a \in \{0,1\}^\lambda$. Therefore, the states sent by Bob to Alice, depending on the parity of a are

$$\rho_{\text{even}} = \frac{1}{2^{2\lambda-1}} \sum_{a \text{ even}} \sum_{x_1,\cdots,x_\lambda} H^{x_1} |a_1\rangle\langle a_1| H^{x_1} \otimes \cdots \otimes H^{x_\lambda} |a_\lambda\rangle\langle a_\lambda| H^{x_\lambda} \tag{24}$$

$$\rho_{\text{odd}} = \frac{1}{2^{2\lambda-1}} \sum_{a \text{ odd}} \sum_{x_1,\cdots,x_\lambda} H^{x_1} |a_1\rangle\langle a_1| H^{x_1} \otimes \cdots \otimes H^{x_\lambda} |a_\lambda\rangle\langle a_\lambda| H^{x_\lambda}. \tag{25}$$

We will bound $\|\rho_{\text{even}} - \rho_{\text{odd}}\|_1$ and therefore bound the probability that Alice successfully guesses the parity.

$$\|\rho_{\text{even}} - \rho_{\text{odd}}\|_1 = \frac{1}{2^{2\lambda-1}} \left\| \sum_a \sum_{x_1,\cdots x_\lambda} (-1)^{a_1\oplus\cdots\oplus a_\lambda} H^{x_1} |a_1\rangle\langle a_1| H^{x_1} \otimes \cdots \otimes H^{x_\lambda} |a_\lambda\rangle\langle a_\lambda| H^{x_\lambda} \right\|_1 \tag{26}$$

$$= \frac{1}{2^{2\lambda-1}} \left\| \left(\sum_{a_1,x_1} (-1)^{a_1} H^{x_1} |a_1\rangle\langle a_1| H^{x_1} \right) \otimes \cdots \otimes \left(\sum_{a_\lambda,x_\lambda} (-1)^{a_\lambda} H^{x_\lambda} |a_\lambda\rangle\langle a_\lambda| H^{x_\lambda} \right) \right\|_1 \tag{27}$$

$$\leq \frac{1}{2^{2\lambda-1}} \left\| \sum_{a_1,x_1} (-1)^{a_1} H^{x_1} |a_1\rangle\langle a_1| H^{x_1} \right\|_1^\lambda \tag{28}$$

$$= \frac{1}{2^{2\lambda-1}} \left\| \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \right\|_1^\lambda \tag{29}$$

$$\leq \frac{1}{2^{2\lambda-1}} (2\sqrt{2})^\lambda \tag{30}$$

$$= 2 \left(\frac{\sqrt{2}}{2} \right)^\lambda \tag{31}$$

Therefore, the probability of successfully cheating is $p^{s=1}_{*b} = \frac{1}{2} + \frac{\|\rho_{\text{even}} - \rho_{\text{odd}}\|_1}{4} \leq \frac{1}{2} + \frac{1}{2}(\frac{1}{\sqrt{2}})^\lambda$. □

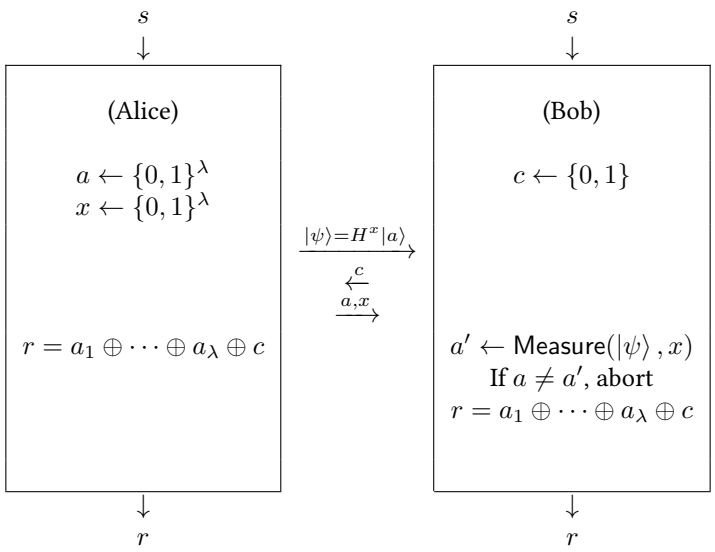


Figure 11: Commitment-based protocol for the case $s = 0$. When $s = 1$ the roles of Alice and Bob are switched.

Protocol	s	(p^s_{0*}, p^s_{1*})	$(p^s_{*0}, p^s_{*,1})$	Remarks
Dummy Protocol (see Algorithm 32)	0	$(1,0)$	$(0,1)$	
	1	$(0,1)$	$(0,1)$	
				$\varepsilon_A = \varepsilon_B = \frac{1}{2}$ $\epsilon_A = \epsilon_B = 0.$
The s-flips protocol (see Algorithm 33)	0	$(1/2, 1/2)$	$(1,1)$	
	1	$(1,1)$	$(1/2, 1/2)$	
				$\varepsilon_A = \varepsilon_B = \frac{1}{4}$ $\epsilon_A = \epsilon_B = \frac{1}{4}.$

Table 1: All protocols above are *correct*, i.e. $p_{00} = p_{11} = 1/2$ and $p_{01} = p_{10} = 0$.

7.3 Coin flipping in the shared-EPR model

Finally, in this section we look at coin flipping protocols in the shared-EPR model. We restrict to "measure-and-compute" protocols, i.e. protocols that are classical except that they are allowed to measure their part of the shared EPR states in the standard of Hadamard basis, at any step during their execution. There are no limitations on the adversaries, however—they can perform arbitrary quantum computations on their shares of EPR pairs. These restricted protocols are of interest because they are consistent with the device independent approach, i.e. any measure-and-compute protocol $\mathcal{P}$ in the shared-EPR setting can be realised by first running 2-FAST in the offline phase to share EPR states (up to isometries), and then executing protocol $\mathcal{P}$ in the online phase, device independently.

Definition 38 (Measure-and-compute coin flipping protocol (in the shared-EPR model))**.** *A measure-and-compute coin flipping protocol is specified by*

- *two functions* $f : \mathbb{N} \to \mathbb{N}, k : \mathbb{N} \to \mathbb{N}$*, and*
- *two interacting machines* A, B
  - *each of which takes two inputs, a security parameter as a unary string* 1^λ*, and* $k(\lambda)$ *qubits, and*
  - *interact to produce output* a, b *respectively, denoted by* $(a, b) \leftarrow \langle A, B\rangle$*, in time at most* $f(\lambda)$ *where* $a, b \in \{0, 1\}$*.*

The machines A *and* B *are classical except that at any step during their execution, they can measure any of their qubits in either the standard or the Hadamard basis, and use the result in subsequent steps. No limits are imposed on* f*.*

Let p_{ab}, p_{a*}, p_{*b} be as in Notation 25 where B' and A' are arbitrary quantum machines. The notion of bias ϵ remains unchanged. However, the notion of predictability bias needs to be extended to the shared-EPR setting from the shared-randomness setting. The idea remains the same, we want to know, if before interacting with the honest party, the malicious party can interact with its qubit to learn a string τ, such that given τ, it is able to predict the outcome of the honest party. (The malicious party is allowed is allowed to deviate in any arbitrary way, conditioned on τ, during the execution of the protocol.) We formalise this as follows.

Notation 39. Fix any measure-and-compute coin flipping protocol (in the shared-EPR model as in Definition 38) and the security parameter λ. Denote by

- $p^\tau_{*b} := \max_{A'} \sum_{a'} \Pr\left[(a', b) \leftarrow \langle A'_1, B\rangle (\Psi^\tau_{MQ})\right]$ where $A' := (A'_0, A'_1)$ is a pair of quantum interactive machine (see Figure 12)
  - with A'_0 taking as input a k-qubit register P, producing a classical string τ in register T and a state in a quantum register M,
  - with Ψ^τ_{MQ} being the joint state after A'_0 acts on P and where Q is the register holding the other half of EPR pairs, conditioned on register T containing τ (which means Ψ^τ_{MQ} is normalised) and
  - with A'_1 taking as input the register M, and producing output (a', b) by interacting with machine B.
- $p^\tau_{a*} := \max_{B'} \sum_{b'} \Pr[(a, b') \leftarrow \langle A, B'_1\rangle (\Psi^\tau_{PM})]$ where $B' := (B'_0, B'_1)$ is defined analogously, to wit: a pair of quantum interactive machine
  - with B'_0 taking as input a k-qubit register Q, producing a classical string τ on register T and a state in a quantum register M,
  - with Ψ^τ_{PM} being the joint state after B'_0 acts on Q where P is the register holding the other half of EPR pairs, conditioned on register T containing τ (which means Ψ^τ_{PM} is normalised) and
  - with B'_1 taking as input the register M, and producing output (a, b') by interacting with machine A.

Predictability bias. Denote Alice's predictability bias given τ, by

$$\varepsilon^\tau_A := \max_b p^\tau_{*b} - \frac{1}{2}, \text{ and by } \varepsilon_A := \langle \varepsilon^\tau_A \rangle_\tau$$

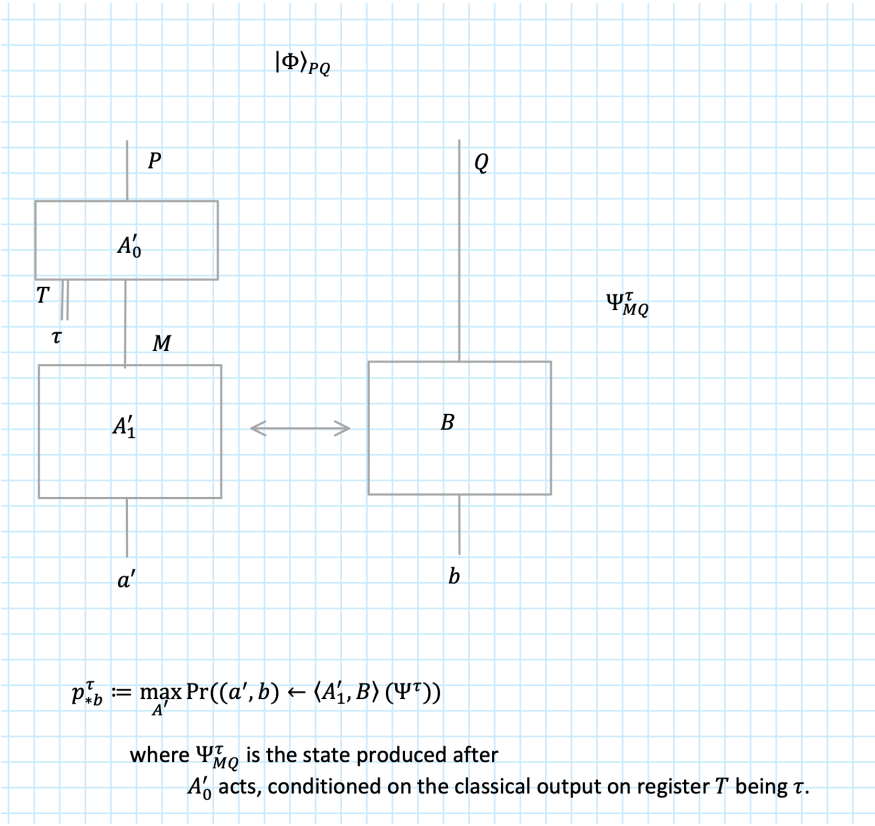

$|\Phi\rangle_{PQ}$
P
Q
A'_0
T
τ
M
Ψ^{τ}_{MQ}
A'_1
B
a'
b
$p^{\tau}_{*b} \coloneqq \max_{A'} \Pr((a', b) \leftarrow \langle A'_1, B \rangle (\Psi^{\tau}))$
where Ψ^{τ}_{MQ} is the state produced after
A'_0 acts, conditioned on the classical output on register T being τ.

Figure 12:

her predictability bias (where the average is over the probability of obtaining τ). Similarly, Bob's predictability bias given τ is

$$\varepsilon_B^\tau := \max_a p_{a*}^\tau - \frac{1}{2}, \text{ and } \varepsilon_B := \langle \varepsilon_B^\tau \rangle_\tau .$$

Predictability bias is then given by $\varepsilon := \max\{\varepsilon_A, \varepsilon_B\}$.

We claim that the following protocol has vanishing predictability bias.

Algorithm 40 (The measure-and-XOR protocol)**.** *Fix $k(\lambda) = 2\lambda$ and let*

$$|\Psi\rangle_{AA'} := |\Phi\rangle_{A_1A_1'} \otimes \cdots \otimes |\Phi\rangle_{A_\lambda A_\lambda'} \text{ and } |\Psi\rangle_{BB'} := |\Phi\rangle_{B_1B_1'} \otimes \cdots \otimes |\Phi\rangle_{B_\lambda B_\lambda'}$$

where $|\Phi\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$. Alice receives the k-qubit register AB and Bob receives the k-qubit register $A'B'$. (This just sets up some notation for how the EPR pairs are shared) and proceed as in Figure 13.

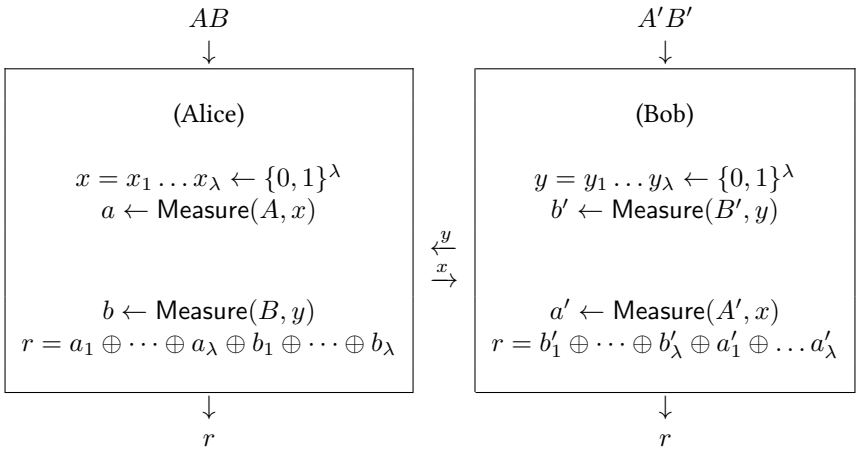


Figure 13: Measure-and-XOR protocol

Theorem 41. *There is a non-increasing vanishing function $\varepsilon' : \mathbb{N} \to \mathbb{R}^+$ such that Algorithm 40 has predictability bias (as in Notation 39) $\varepsilon \leq \varepsilon'$, which in particular means $\lim_{\lambda\to\infty} \varepsilon(\lambda) = 0$.*

[TODO: I'll need to find an analytic solution to be able to claim this, otherwise I'll have to claim a numerical value, which is less nice]

7.3.1 Computing predictability of measure-and-compute is an SDP

Consider a general 4-message coin flipping protocol in the shared-EPR model as shown in Figure 14 where . We start with recalling (from prior works) that the bias for this protocol can be cast into a semi-definite programme (SDP) as follows: $\epsilon_A + \frac{1}{2}$ is at most

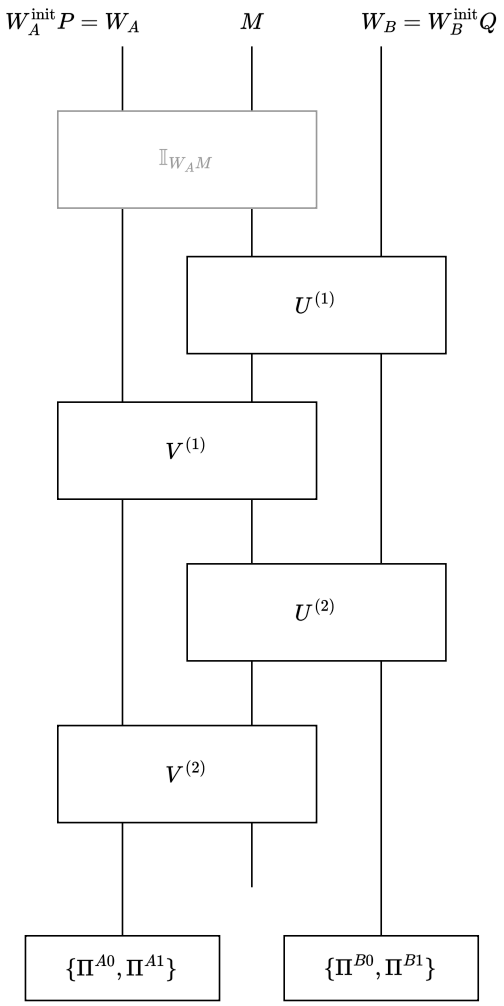
$$\max_{r\in\{0,1\},\rho^{(1)}\rho^{(2)}} \left[\mathrm{tr}\left(|r\rangle\langle r|_{R_S} \left(U^{(2)}\rho^{(1)}_{MW_B}U^{(2)\dagger}\right)\right)\right]$$

subject to

$$\begin{aligned} \rho^{(1)}, \rho^{(2)} &\geq 0 \\ \mathrm{tr}(\rho^{(1)}) = \mathrm{tr}(\rho^{(2)}) &= 1 \\ \rho^{(1)}_{MW_B} &= \left|\Psi^{\mathsf{initA}}\right\rangle \left\langle\Psi^{\mathsf{initA}}\right|_{MW_B^{\mathrm{init}}} \otimes \frac{\mathbb{I}_Q}{2} \\ \mathrm{tr}_M(\rho^{(2)}_{MW_B}) &= \mathrm{tr}_M(U^{(1)}(\rho^{(1)}_{MW_B})U^{(1)\dagger}). \end{aligned}$$

TODO: Explain why these constraints.

(a) Protocol



(b) Computing Bob's bias ϵ_B

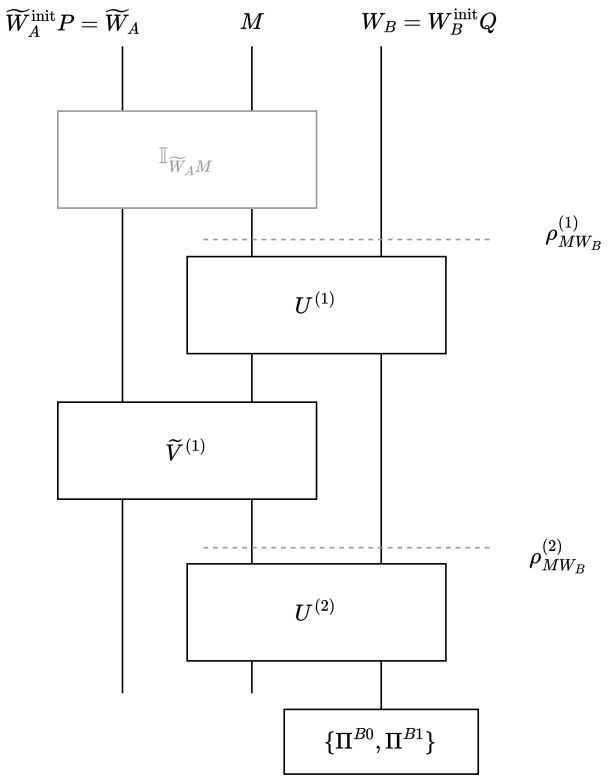


Figure 14:

(c) Computing Bob's predictability ε_B

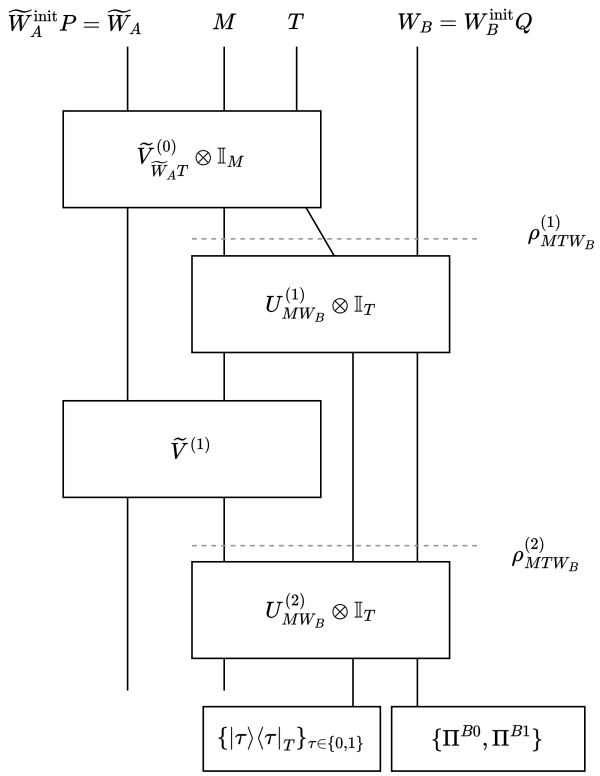


Figure 15:

Moving to the case of unpredictability, we formalise the prediction made by a malicious Bob as it applying an operation $\tilde{V}^{(0)}_{\tilde{W}_A T}$ to obtain a prediction in the register T. To make the analysis cleaner, we simply give register T to the honest party who is asked to leave the register unaffected until the very last step. At the last step, the honest party, in addition to producing the outcome r, also measures register T to obtain τ. The predictability condition is then that $\tau = r$. Proceeding as in the previous case, one can thus cast predictability as an SDP as follows.

TODO: Relax constraint 1 to be $\text{tr}_{TM}(\rho^{(1)}_{MTW_B})$ instead of tracing only T. This will give the adversary more control and allow me to use the bound more generally.

Proposition 42. *Consider a general 4-message coin flipping protocol $\mathcal{P}$ in the shared-EPR model as shown in Figure 14. Denote by ε_B the predictability of Bob in protocol $\mathcal{P}$. Using the notation in Figure 15, we assert that $\varepsilon_B + \frac{1}{2} \leq v^{primal}$ is at most the value v^{primal} of the following (primal) SDP*

$$v^{primal} := \max_{\rho^{(1)},\rho^{(2)}} \left[\text{tr} \left[\left(|0\rangle\langle 0| \otimes \Pi^{B0} + |1\rangle\langle 1| \otimes \Pi^{B1} \right)_{TW_B} U^{(2)}_{MW_B} \rho^{(2)}_{MTW_B} U^{(2)\dagger}_{MW_B} \right] \right]$$

subject to

$$\begin{aligned} \rho^{(1)}, \rho^{(2)} &\in \mathsf{DensityMatrix}(MTW_B), \\ \text{tr}_T(\rho^{(1)}_{MTW_B}) &= \left|\psi^{\mathsf{initMW_B}}\right\rangle \left\langle\psi^{\mathsf{initMW_B}}\right|_{MW_B^{\text{init}}} \otimes \frac{\mathbb{I}_Q}{2}, \\ \text{tr}_M(\rho^{(2)}_{MTW_B}) &= \text{tr}_M(U^{(1)}_{MW_B} \rho^{(1)}_{MTW_B} U^{(1)\dagger}_{MW_B}). \end{aligned}$$

Furthermore, $\varepsilon_B + \frac{1}{2} \leq v^{dual}$ is at most the value v^{dual} of the corresponding dual SDP:

$$v^{dual} := \min_{Z^{(1)}, Z^{(2)}} \operatorname{tr}\left[Z^{(1)}_{MW_B} \left(\left|\psi^{\mathsf{initMW_B}}\right\rangle \left\langle\psi^{\mathsf{initMW_B}}\right|_{MW_B^{init}} \otimes \frac{\mathbb{I}_Q}{2} \right) \right]$$

subject to

$$\begin{aligned}
Z^{(1)} &\in \mathsf{Pos}(MW_B), \\
Z^{(2)} &\in \mathsf{Pos}(TW_B), \\
Z^{(1)}_{MW_B} \otimes \mathbb{I}_T &\geq U^{(1)\dagger}_{MW_B} (Z^{(2)}_{TW_B} \otimes \mathbb{I}_M) U^{(1)}_{MW_B}, \\
Z^{(2)}_{TW_B} \otimes \mathbb{I}_M &\geq U^{(2)\dagger}_{MW_B} \left(\left(|0\rangle\langle 0| \otimes \Pi^{B0} + |1\rangle\langle 1| \otimes \Pi^{B1} \right)_{TW_B} \otimes \mathbb{I}_B \right) U^{(2)}_{MW_B}.
\end{aligned}$$

Proof. The fact that the primal SDP upper bounds $\varepsilon_B + \frac{1}{2}$ can be seen by inspection (recall Notation 39 and the corresponding discussion above for ϵ_A). We derive the dual using standard techniques. Denote by $\rho = \{\rho^{(1)}, \rho^{(2)}\}$ and by $Z = \{Z^{(1)}, Z^{(2)}\}$. Let $\Pi := \left(|0\rangle\langle 0| \otimes \Pi^{B0} + |1\rangle\langle 1| \otimes \Pi^{B1} \right)_{TW_B}$. Consider the following objective function

$$\begin{aligned}
L(\rho, Z) :=& \operatorname{tr}(\Pi U^{(2)} \rho^{(2)}_{MTW_B} U^{(2)\dagger}) \\
&+ \operatorname{tr}_{MW_B}\left[Z^{(1)}_{MW_B} \left(\left|\psi^{\mathsf{initMW_B}}\right\rangle \left\langle\psi^{\mathsf{initMW_B}}\right|_{MW_B^{\mathrm{init}}} \otimes \frac{\mathbb{I}_Q}{2} - \operatorname{tr}_T(\rho^{(1)}_{MTW_B}) \right) \right] \\
&+ \operatorname{tr}_{TW_B}\left[Z^{(2)}_{TW_B} \left(\operatorname{tr}_M \left(U^{(1)} \rho^{(1)}_{MTW_B} U^{(1)\dagger} - \rho^{(2)}_{MTW_B} \right) \right) \right]
\end{aligned}$$

and note that $v^{\mathrm{primal}} \leq \max_{\rho \geq 0} L(\rho, Z)$ because of the following two reasons:[14] first, the optimal solution ρ^* to the primal problem saturates the inequality and second, maximisation over ρ can only increase the value of $L(\rho, Z)$. As the bound holds for all Zs, the best bound is obtained by minimising over all Zs. We are thus interested in the function $v^{\mathrm{primal}} \leq \min_Z \max_{\rho \geq 0} L(\rho, Z)$. We therefore re-express $L(\rho, Z)$ collecting all dependence on ρ. Why this helps will become clearly shortly. We have

$$\begin{aligned}
L(\rho, Z) = \operatorname{tr}\Bigg[& (\Pi_{TW_B} \otimes \mathbb{I}_M) U^{(2)} \rho^{(2)}_{MTW_B} U^{(2)\dagger} \\
& + Z^{(1)}_{MW_B} \otimes \mathbb{I}_T \left(\left|\psi^{\mathsf{initMW_B}}\right\rangle \left\langle\psi^{\mathsf{initMW_B}}\right|_{MW_B^{\mathrm{init}}} \otimes \frac{\mathbb{I}_2}{2} - \rho^{(1)}_{MTW_B} \right) \\
& + Z^{(2)}_{TW_B} \otimes \mathbb{I}_M \left(U^{(1)} \rho^{(1)}_{MTW_B} U^{(1)\dagger} - \rho^{(2)}_{MTW_B} \right) \Bigg]
\end{aligned}$$

which after pulling out the ρs, becomes

$$\begin{aligned}
L(\rho, Z) = \operatorname{tr}\Bigg[& Z^{(1)}_{MW_B} \left(\left|\psi^{\mathsf{initMW_B}}\right\rangle \left\langle\psi^{\mathsf{initMW_B}}\right|_{MW_B^{\mathrm{init}}} \otimes \frac{\mathbb{I}_Q}{2} \right) \\
& + \rho^{(2)}_{MTW_B} \underbrace{\left(U^{(2)\dagger}_{MW_B} (\Pi_{TW_B} \otimes \mathbb{I}_M) U^{(2)}_{MW_B} - Z^{(2)}_{TW_B} \otimes \mathbb{I}_M \right)}_{\mathrm{I}} \\
& + \rho^{(1)}_{MW_B} \underbrace{\left(U^{(1)\dagger}_{MW_B} (Z^{(2)}_{TW_B} \otimes \mathbb{I}_M) U^{(1)}_{MW_B} - Z^{(1)}_{MW_B} \otimes \mathbb{I}_T \right)}_{\mathrm{II}} \Bigg].
\end{aligned}$$

Now since we maximise L over all $\rho \geq 0$, to obtain a non-trivial value for L, we must have I and II be ≤ 0. We therefore have

$$v^{\mathrm{primal}} \leq \min_Z \operatorname{tr}\left[Z^{(1)}_{MW_B} \left(\left|\psi^{\mathsf{initMW_B}}\right\rangle \left\langle\psi^{\mathsf{initMW_B}}\right|_{MW_B^{\mathrm{init}}} \otimes \frac{\mathbb{I}_Q}{2} \right) \right] =: v^{\mathrm{dual}}$$

[14] By $\rho \geq 0$ we mean $\rho^{(1)} \geq 0$ and $\rho^{(2)} \geq 0$.

subject to

$$Z^{(1)}_{MW_B} \otimes \mathbb{I}_T \geq U^{(1)\dagger}_{MW_B}(Z^{(2)}_{TW_B} \otimes \mathbb{I}_M)U^{(1)}_{MW_B}$$
$$Z^{(2)}_{TW_B} \otimes \mathbb{I}_M \geq U^{(2)\dagger}_{MW_B}(\Pi_{TW_B} \otimes \mathbb{I}_M)U^{(2)}_{MW_B}.$$

Note that since $\Pi_{RT} \geq 0$ (it is a projector), from the last constraint it follows that $Z^{(2)} \geq 0$ and from the first constraint it follows[15] that $Z^{(1)}_{MW_B} \geq 0$. This concludes the proof. □

7.3.2 Stating the measure-and-XOR protocol in the SDP formalism

We write the measure-and-XOR protocol as follows. To this end, we first, recall that $W_A := W_A^{\text{init}}P$ and $W_B := W_B^{\text{init}}Q$. By a λ-qubit register A, for instance, we mean that $A = A_1 \otimes \cdots \otimes A_\lambda$ where each A_i holds a qubit. Suppose the security parameter is fixed to be λ.

- $P := AB$ and similarly, $Q := A'B'$ where A, B, A', B' are all λ-qubit registers.
  - Registers AA' will be use to hold λ EPR pairs and BB' will also be used to hold another λ EPR pairs.
- $W_A^{\text{init}} := X\tilde{X}$ (later we drop $\tilde{A}\tilde{B}$ and keep only R) and similarly $W_B^{\text{init}} := Y\tilde{Y}$ where all registers are λ qubit registers.
  - Here, $X\tilde{X}$ will also be used to contain EPR pairs, but these correspond to the (private) choice made by Alice; one could use a mixed state but then when we analyse it as an SDP, the malicious party can hold a purification and therefore the randomness becomes public. To ensure the randomness stays private, we have Alice hold the purification. This is just to simplify the analysis—in reality, we simply assume Alice can sample private randomness. Similarly for Bob and $Y\tilde{Y}$.

Thus, W_A and W_B both store 3λ qubits, while M is a λ qubit register.

Denote by

$$\left|\psi^{(0,0)}\right\rangle_{W_AMW_B} = \frac{1}{\sqrt{2^n}} \sum_{i\in\{0\ldots 2^\lambda-1\}} \underbrace{|\psi\rangle_{X\tilde{X}}|i\rangle_P}_{|\psi^{\text{initA}}\rangle_{W_A^{\text{init}}P}=|\psi^{\text{initA}}\rangle_{W_A}} |0\rangle_M \underbrace{|i\rangle_Q|\psi\rangle_{Y\tilde{Y}}}_{|\psi^{\text{initB}}\rangle_{W_B}=|\psi^{\text{initB}}\rangle_{QW_B^{\text{init}}}}.$$

The protocol is then specified by the following unitaries:

- "Bob sends y to Alice":

  $$U^{(1)}_{MW_B} = \mathsf{SWAP}_{\tilde{Y}M},$$

  and therefore

  $$\left|\psi^{(0,1)}\right\rangle := U^{(1)}_{MW_B}\left|\psi^{(0,0)}\right\rangle = \frac{1}{2^n}\sum_{i,y\in\{0\ldots 2^\lambda-1\}} \underbrace{|\psi\rangle_{X\tilde{X}}|i\rangle_P}_{\text{Alice's local registers}} |y\rangle_M \underbrace{|i\rangle_Q|y\rangle_Y|0\rangle_{\tilde{Y}}}_{\text{Bob's local registers}}.$$

- "Alice receives y and sends x to Bob":

  $$V^{(1)}_{W_AM} = \mathsf{SWAP}_{\tilde{X}M}$$

  and thus

  $$\left|\psi^{(1,1)}\right\rangle := V^{(1)}_{W_BM}\left|\psi^{(0,1)}\right\rangle = \frac{1}{2^{3(n/2)}}\sum_{i,x,y\in\{0,\ldots 2^{\lambda-1}\}} \underbrace{|xy\rangle_{X\tilde{X}}|i\rangle_P}_{\text{Alice's local registers}} |x\rangle_M \underbrace{|i\rangle_Q|y\rangle_Y|0\rangle_{\tilde{Y}}}_{\text{Bob's local registers}}.$$

[15] This is because if $\mathsf{Mat}_1 \geq 0$ and $\mathsf{Mat}_2 \geq U\mathsf{Mat}_1U^\dagger$ then $\mathsf{Mat}_2 \geq 0$ as well.

- "Bob receives x":

$$U^{(2)}_{MW_B} = \mathsf{SWAP}_{M\tilde{Y}}$$

and so

$$\left|\psi^{(1,2)}\right\rangle := U^{(2)}_{MW_B}\left|\psi^{(1,1)}\right\rangle = \frac{1}{2^{3(n/2)}}\sum_{i,x,y\in\{0,\dots 2^{\lambda-1}\}} \underbrace{|x\rangle_X |y\rangle_{\tilde{X}} |i\rangle_P}_{\text{Alice's local registers}} |0\rangle_M \underbrace{|i\rangle_Q |y\rangle_Y |x\rangle_{\tilde{Y}}}_{\text{Bob's local registers}}.$$

- "Alice and Bob, measure and XOR":
  - For notational simplicity, let $X_{\text{copy}} := \tilde{Y}$ and $Y_{\text{copy}} := \tilde{X}$ because $\tilde{X}$ is supposed to contain a copy of y in register Y and $\tilde{Y}$ is supposed to contain a copy of x in register X.
    In this notation, we have

$$\left|\psi^{(1,2)}\right\rangle = \frac{1}{2^{3(n/2)}}\sum_{i,x,y\in\{0,\dots 2^{\lambda-1}\}} \underbrace{|x\rangle_X |y\rangle_{Y_{\text{copy}}} |i\rangle_P}_{\text{Alice's local registers}} |0\rangle_M \underbrace{|i\rangle_Q |y\rangle_Y |x\rangle_{X_{\text{copy}}}}_{\text{Bob's local registers}}.$$

  - Let

$$\Pi := \sum_{x,y\in\{0,1\}^\lambda, s\in\{0,1\}^{2\lambda}:\mathsf{XOR}(s)=1} |x\rangle\langle x| \otimes |y\rangle\langle y| \otimes H^x H^y |s\rangle\langle s| H^x H^y$$

    and define $\Pi^A := \Pi_{XY_{\text{copy}}P}$ and $\Pi^B := \Pi_{X_{\text{copy}}YQ}$.

7.4 Towards simulation security for coin flipping

Part II
Device Independent Multi-Party Computation | DI MPC

8 DI MPC definition

9 Relation to prior work

10 Protocol for DI MPC with quantum communication

11 Definition m-FAST

12 Protocol for m-FAST under a compiled rigidity conjecture

References

[Amb04] Andris Ambainis. A new protocol and lower bounds for quantum coin flipping. *Journal of Computer and System Sciences*, 68(2):398–416, 2004. Special Issue on STOC 2001.

[CLOS02] Ran Canetti, Yehuda Lindell, Rafail Ostrovsky, and Amit Sahai. Universally composable two-party and multi-party secure computation. In *Proceedings of the Thiry-Fourth Annual ACM Symposium on Theory of Computing*, STOC '02, pages 494–503, New York, NY, USA, 2002. Association for Computing Machinery.

[HSS15] Sean Hallgren, Adam Smith, and Fang Song. Classical cryptographic protocols in a quantum world. *International Journal of Quantum Information*, 13(04):1550028, 2015.